UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat d'Informàtica de Barcelona

FIB

DYNAMIC 3D EROSION OF TERRAINS

LLUNA CLAVERA COMAS

Thesis supervisor
OSCAR ARGUDO MEDRANO (Department of Computer Science)

Degree
Master's Degree in Innovation and Research in Informatics (Computer Graphics and Virtual Reality)

Master's thesis

Facultat d'Informàtica de Barcelona (FIB)

Universitat Politècnica de Catalunya (UPC) - BarcelonaTech

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Virtual terrains appear in a wide variety of applications: from entertainment to physical simulation or science visualization, a lot of virtual scenes are set on a digital landscape. Despite the ubiquity of virtual terrains on 3D scenes, the complex and fractal structure of natural landscapes still brings multiple challenges for the field of synthetic terrain generation.

Most virtual terrains are represented using 2D heightfields which, despite being highly convenient when generating, storing and manipulating terrain models, have a major drawback: heightfields can only represent 2.5D models. Because each point in a 2D plane has a set elevation and every point between the ground and the given elevation is considered as solid (and the terrain interior), we can not represent features like caves, arches or overhangs, where not all points between the ground and the maximum elevation are solid. Additionally, these representations of terrains also fail to reproduce complex geometry commonly found in rocky terrains, like sharp ridges, loose blocks or steep cliffs.

Many of these complex structures that can not be easily captured by 2D heightfields, arise from different types of weathering events, like percolation and freeze-thaw cycles. To address this limitations on virtual terrain models, our goal in this thesis is to study the mechanical weathering of rocky terrains to enhance models given from 2D heightfields and add the complex features that these fields fail to capture.

Terrain enhancement through weathering simulation not only improves the realism and quality of the models, adding the aforementioned complex features, but it can also be applied dynamically to simulate the changes a terrain goes through over multiple years.

This project is built on a previous project on this same topic [1]. Using the same algorithm as a base, we worked on addressing some of its limitations and improving both its quality and efficiency.

1.1 Contributions

As we just mentioned, this project is heavily based on a previous one, and our proposed algorithm is built on the method they proposed. Despite that, in this thesis we will provide multiple improvements to the base algorithm to address some of its limitations.

The main point we wanted to address of the previous project was the lack of a physically based approach to the algorithm, as the proposed algorithm made a lot of simplifications and ignored multiple

physical interactions which made the method less accurate and realistic. However, during this thesis we also found other areas that could be improved and expanded, which also took a significant ammount of effort to solve.

The limitations of the previous project and our improvements will be discussed at length during this thesis, but our main contributions can be summarized in two large blocks: improvements to the decomposition of the terrain into cells and improvements on the quality of the algorithm by including a mechanical model to the simulation.

1.2 Set-up and tools

Another difference between the project we base this thesis on, albeit less significant, resides in the tools used. In the project that presented the algorithm we are extending, a blender plugin was implemented to show the method, however, here we have implemented our erosion simulation algorithm using C++ and we showed the results in a QT application using OpenGL for rendering. All deteils of our application will be described in 6, but we also leave all implemented code open to everyone in the Github page of the project

To make the application, we reused the code provided by an article on terrain descriptors [2], as it already implemented many basic functionalities to work with virtual terrains. We then modified and expanded the application implementing our algorithm for mechanical weathering as well as implementing other functionalities to aid in the visualization and debugging of our method. Some of the changes that we implemented modify the basic functionalities of the program, so our project should not be seen as an extension to said application.

Finally, we used the C++ library Voro++ [3] to generate voronoi decompositions, a core aspect of our erosion algorithm.

Chapter 2

Preliminaries

To understand this project and our contributions to the field, we will first present some background information about the topic, so that the reader can better understand both the problem and our proposed solutions.

2.1 Terrains and Landscapes

T

2.1.1 Freeze-thaw cycle

2.2 Terrain Representation and Modeling

Virtual terrains are a fundamental aspect of this thesis, so it is important to review current approaches to represent and work with these types of models. A deep dive into the state-of-the-art on terrain modeling can be found on [4], but we are going to present only the most relevant methods for our project, as a full review is out of the scope of this thesis.

2.2.1 Terrain Representations

When representing terrains and landscapes there are multiple data structures and representations we can use depending on the amount of information that we need to store and the nature of the model.

The main model representation techniques can be divided in *Elevation Models*, mainly focused on representing the surface of the terrain; *Volumetric Models*, that aim to capture some internal properties of the terrain as well as more complex structures; and finally *Implicit Surface Reprsentations* which can be calssified as volumetric models but have some interesting properties that are important to highlight.

Elevation Models

Elevation models are the simplest ways of representing virtual terrains. They consist on assigning an height to every point on a 2D space. In a more formal way, a terrain is represented by a function $h : \mathbb{R}^2 \to \mathbb{R}$ that is at least C^0. From a height function we can easily extract the slope at one point $s(x, y)$ by using the norm of the gradient:

$$s(x,y) = \|\nabla h(x,y)\| = \left\|(\frac{\partial h}{\partial x}, \frac{\partial h}{\partial y})\right\| \tag{2.1}$$

And from the gradient (projected to $\mathbb{R}^3$) we can easily obtain the normal at one point $n(x,y)$:

$$n(x,y) = \frac{(-\frac{\partial h}{\partial x}, -\frac{\partial h}{\partial y}, 1)}{\left\|(-\frac{\partial h}{\partial x}, -\frac{\partial h}{\partial y}, 1)\right\|} \tag{2.2}$$

With the height, normal and slope we have all that we need for a basic representation of a terrain. Of course, as we are using a bivariate function to represent the terrain, we can only have one elevation for each point on the domain, and that makes features like caves or overhangs impossible to represent with this method.

Despite its limitations, elevation models are widely used for its simplicity and memory efficiency as they can capture the general structure of a landscape, given that complex features like overhangs often represent a small part of the overall terrain.

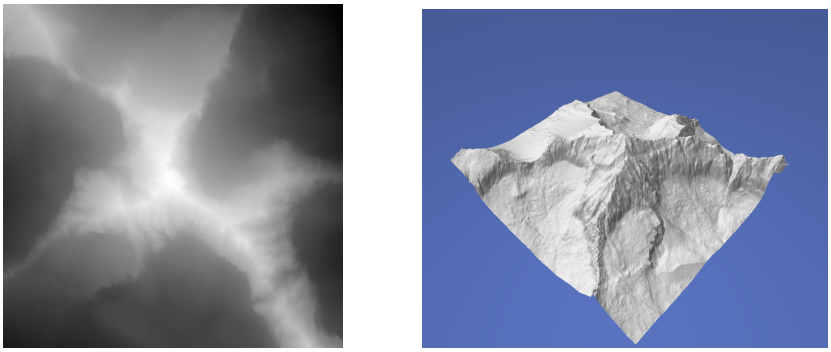
To expand on this topic, elevation models only require a bivariate function to represent the whole terrain, which can capture the whole terrain in a really simple closed-form expression, providing infinite precision with an extremely small amount of memory. However, constructing functions that represent complex realistic terrains is highly challenging and, for that reason, elevation models commonly use *discrete heightfields* (also called *Digital Elevation Models* or DEM) to represent terrains.

Discrete Heightfields are a collection of altitudes in a 2D grid. In order to create a DEM, you have to divide the domain of the terrain (assuming a rectangular domain) into a grid of $n \times m$ cells, then, you assign an altitude to every cell of the grid. The grid will only contain concrete values for a set of discrete values, but you can obtain a height for every point of the domain by interpolating multiple values of the heightfield. This method limits the accuracy of the model (as we stop having infinite precision), but it has the advantadge of representing arbitrary terrains in a really simple way.

Heightfields are an extremely common way to represent terrains, and they can be easily stored in textures and monochromatic images (as figure 2.1 shows), sometimes making use of the texture interpolation capabilities on the GPU to accelerate computations on heightfields. Specifically, DEMs are used on [2], a project focused on terrain analysis that provides multiple metrics for terrains and whose code has been reutilized in this project to implement and test our erosion algorithm.

To finalize the topic of elevation models, they have been extended to represent some internal structure of the model by employing a layered representation of the terrain. Layered representations are basically an ordered collection of height functions, each function representing a layer of the model (which usually corresponds to a different material), resulting in a stack of elevations for each grid cell. A representation of these types of layered heightfields can be seen on figure 2.2

Layered representations have been used to simulate erosion ([5] and [6]) and, specially to simulate weathering events related to sediments as those types of rocks are naturally organized in layers. There are interesting works on these topics, but those will be covered on chapter 3



(a) Aneto heightfield (b) Aneto model on the app

Figure 2.1: Heightfield of the mountain Aneto as a monochromatic image and the terrain produced by said heightfield on our application

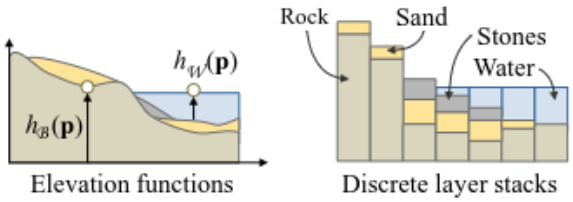


Figure 2.2: Illustration of stacked height functions, figure taken from [4]

Volumetric Models

As seen previously, layered representations are a way to represent different materials and properties of a terrain, but if we are really interested in encoding the internal structure of a landscape, we will have to use volumetric models. Volumetric models can be defined by a function $\mu : \mathbb{R}^3 \to \mathcal{M}$, where $\mathcal{M}$ denotes the set of materials used (air can count as a material). This function will be assigning every point of the 3D domain to a material, so now complex structures like overhangs can be represented, as well as different internal compositions of the terrain.

Despite volumetric models being defined with a function, in practice we will usually use a 3D grid with discrete samples and perform some interpolation to obtain values outside the fixed grid points, like we do with DEMs. Each cell of the 3D grid is usually called a *voxel* and the transformation of a model into a set of voxels a *voxelization*. Voxelizations allow us to freely move and remove parts of the model, so they have potential to be used for algorithms that change the structure of the landscape, like for erosion simulation, and they have been explored in the literature [7], but they also have the huge drawback of being extremely expensive memory-wise. For that reason there has been a lot of work in the literature making voxel representations more efficient (like adaptative grids) or presenting novel ways to represent models encoding volumetric information.

For our purposes, we could have used a voxelization to implement our algorithm, but we decided that decomposing the terrain in voronoi cells would produces more natural results in a more efficient manner. The main concepts of the decomposition that we have chosen will be explained on section 2.3.2.

Implicit Surfaces

From a field function $f : \mathbb{R}^3 \to \mathbb{R}$ over the domain Ω of the terrain you can implicity define a surface the following way:

$$\mathcal{S} = \{p \in \mathbb{R}^3 | f(p) = 0\} \tag{2.3}$$

That is, the surface would contain the set of all points which map to 0 in the scalar field f. Going from a heightfield h to an implicit surface is really straight-forward (although the opposite is not), you just need to define field f as $f(p) = h(p_{xy}) - p_z$, from that you can go to a usual volumetric representation, defining μ as $\mu(p) = 1$ if $f(p) < 0$ and $\mu(p) = 0$ otherwise (meaning the material will be rock if and only if the function f is negative) for example.

Scalar fields defined from elevation functions are a type of distance fields: for every point of the 3D space the field represents the vertical distance to the surface, being negative if the point is inside and positive otherwise. You can also use other distance fields to define surfaces (with any definition of distance that fits your situation) and you could easily transform those distance fields into standard volumetric models by assigning different distances to different materials.

2.2.2 Terrain Synthesis

Terrain generation techniques can be divided in three main groups (although it is common to employ methods from the three groups in the same project):

- **Procedural Generation:** Procedural generation methods artificially generate terrains from scratch by applying randomized processes, like faulting or using noise.
- **Example-based:** Example-based methods create virtual terrains using scanned-data of real-world terrains.
- **Erosion Simulation:** Erosion simulations methods apply algorithms to simulate geomorphological processes when creating the landscape.

Synthetic terrains usually can not represent all the complexity of a natural landscape by procedural generation alone, and that is why erosion simulations are so important, not only in order to "predict" a future for the landscape but also to add realism on virtual scenes. Even example-based terrains often fail to capture some aspects of the terrain they are trying to replicate, as scans have precision limitations (and the bigger the region to scan, the more difficult it is to do it with accuracy), and erosion simulation can also intervene in those cases to add more complexity to the landscapes.

2.3 Model Decompositions

One of the core aspects of simulating erosion on 3D terrains is the model representation that we use. Any erosion simulation applied on a 3D model will modify the model and its properties, so proper data structures that allow for dynamic modifications of the model are key for these types of simulations. Additionally, applying erosion to an object also exposes the *interior* of said object, so any model representation that we use also has to encode some internal information of the model, as internal points of a model might become part of the surface at some point of the process.

Figure 2.3: Example of a tetrahedralization, taken from [9]

In this section we are going to explain some of the decompositions that can be applied to virtual terrains in order to convert the full model into a set of independent parts that can move freely to modify the overall landscape. We already mentioned voxelizations before (on section 2.2.1), but due to voxelization limitations (mainly the memory cost), we explored more complex techniques to decompose our models.

2.3.1 Tetrahedralization

As you might already know, every polygon can be divided into a set of triangles that exactly match the polygon perimeter. Triangulations can also be used to approximate curved surfaces with arbitrary precision. In the case of 3D polyhedra and volumes, tetrahedra are the equivalent of triangles, as we can approximate any volume using terahedra, with more efficiency than with any other polyhedra (like cubes).

We could use a tetrahedralization to decompose our terrain into tetrahedra and then apply our algorithm over those pieces, and we could even further divide the tetrahedra into more tetrahedra to adjust the granularity of the decomposition. The concept of tetrahedralization and tetrahedral meshes has been used to simulate erosion in the past [8], but we have chosen a different decomposition that has some advantages over tetrahedralizations and produce more irregular shapes, which better represents the shapes we can observe in nature.

2.3.2 Voronoi Decomposition

Given a set of sample points, a voronoi decomposition is a partition of the space into multiple regions, where every region is defined by the set of all points that are closer to a given sample than any other sample. In a more formal way, given a set of centroids $C = \{c_1, c_2 \dots c_n\}$, the set of regions defined by those centroids $\mathbf{R} = \{R_1, R_2, \dots R_n\}$ is defined as follows:

$$R_i = \{p \in \Omega | \forall c_j \in C, \;\; d(p, c_i) \leq d(p, c_j)\} \tag{2.4}$$

Where $\Omega \subseteq \mathbb{R}^3$ is the domain we are dealing with (In our case it will be the volume of the model we are subdividing), c_i is the centroid of region R_i and function d represents the euclidean distance. Notice that a point might lie in more than one region, when the distance from that point to multiple centroids is exactly the same. Points lying inside two regions create a plane that defines the boundary between those two regions.

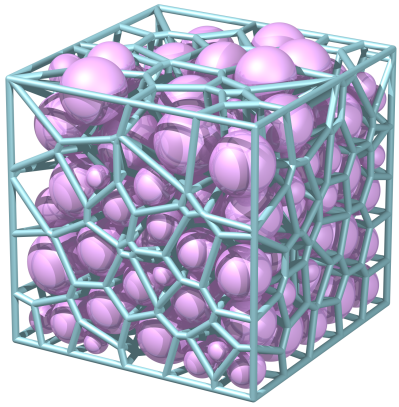


Figure 2.4: Example of a voronoi decomposition of a cube, showing the particles and faces of the decomposition. Figure taken from the documentation of the C++ library Voro++ [3]

In this thesis, we decided to use voronoi cells for multiple reasons. First, they can be generated with a lot of flexibility (just choose an arbitrary set of points), and we can adapt the decomposition depending on our needs. Additionally, the generated cells are not very regular, with varying sizes and shapes, which produces a more natural rocky appearance than other more regular decompositions.

Chapter 3

Previous Work

There has been a multitude of proposed approaches to the problem of erosion simulation, each using different techniques and tackling different parts of the problem. To better understand our method and how it compares to previous solutions, we present a summary of projects in the literature, with their limitations and their differences with our approach. Additionally, we will also discuss the paper we are basing our algorithm on [1], as our project is an extension to their work.

3.1 Work in Applied sciences

3.1.1 Models to Simulate Rock Systems

There have been multiple approaches to simulate the complex mechanics of rocky objects and the interaction between bonds in their internal structure. On this topic, [10] provides an overview on rock mechanics as well as a classification of different computational methods to simulate said mechanics. The article divides the different methods into continuum and discontinuum methods. In our case, the mechanical model that we implemented divides the domain into multiple discrete pieces that can move, rotate, and interact independently, so our method falls into the family of discontinuum methods.

The full family of computational methods, fracture mechanics, and rock behavior simulation is really extense and we are only working with a tiny aspect of this full family. Our project is based on the *Bonded Discrete Element Method*, which is based on the more general *Discrete Element Method*, which is also part of the family of *Discontinuum Methods*, so we are going to focus on this specific branch of mechanical models. Still, for the curious reader, [11] contains an in-depth explanation of the *Finte Element Method*, one of the most popular methods in rock engineering and the basis for multiple methods of the continuum method family.

3.1.2 Discrete Element Method for Rock Simulation

In this thesis, we are proposing a mechanical model based on the *Discrete Element Method*, often abbreviated as *DEM* (it has the same abbreviation as Digital Elevation Models, so, in order to not confuse the reader, we will try not to use both concepts in a single section). In this regard, [12] provides a recent review of multiple advances on the *Discrete Element Method* as well as on other discontinuum methods like the *Lattice Element Method* or the *Particle Replacement Method*. The goal of the article is to improve comminution models, but the methods described are also useful for our purposes.

Figure 3.1: A welsh dragon breaking. Taken from [18]

Among other methods, the article describes the *Bonded Discrete Element Method*, which we are going to use, and its origins ([13]) as well as how voronoi cells are often used ([14], [15]) due to their multiple advantages, like their closer resemblance to the irregular shapes of natural grains.

3.2 Work in Computer Graphics

3.2.1 Related Methods in Computer Graphics

In this section we present work that is not focused on terrain erosion, but it is still somewhat related to the topic.

Spring-Mass Systems and Brittle Fracture

Solid fracture is a fundamental aspect of simulating rock erosion, specially when simulating the fractures caused by the infiltration of water and its freezing. To simulate breakage, one common and simple approach is to use spring-mass systems and remove overly extended springs. This approach was demonstrated years ago in [16] and [17].

A more recent approach on this topic was presented in [18], where they explore the use of peridynamic systems for brittle fracture. Peridynamic systems are spring-mass systems with two particular properties: they decouple stiffness from spring length and they also connect particles just by proximity instead of using 1-ring neighborhoods. An interesting part of the paper is that they have tried using Voronoi-Based Geometry to decompose the model and apply their spring-mass algorithm. In our case, we also use voronoi cells to generate our *masses*, although the dynamics of the links and their fracture are different in our approach. Another approach that uses voronoi geometry for fracture can be found in [19], although in this case they use voronoi polygons over a surface instead of generating voronoi polyhedra in a volume.

Modeling Wood Decay

Modeling realistic trees has some parallels to modeling terrains: The goal is to represent a complex natural object constantly modified through the interaction with its environment. Some work has been done [20] [21] on simulating tree decay, which is the equivalent of simulating erosion on rocky terrains.

On this specific topic, the recent work of [22] (which is yet to be published on SIGGRAPH 2026) presents a really interesting method to simulate wood decay from fungal interaction. The method consists

Figure 3.2: Example of wood dedcay of a cut tree trunk. Figure taken from [22]

in dividing the tree model into *segments* and then simulating a fungus infiltrating the host and generating fractures in the tree segments. A fungal infiltration also creates conditions facilitating further decay, which has similaritites to the problem we are tackling: we divide a terrain into different components, and then we simulate the water infiltrating the rock, fracturing it and creating conditions where more water infiltration is favoured. The specific mechanics of rock fracture and wood decay are very different, so techniques for wood decay are not really applicable to our work, even if there are interesting parallels.

3.2.2 Weathering in Computer Graphics

Here we will show a summary of proposed methods for erosion simulation in the field of computer graphics.

Rock Erosion Simulation with Volumetric Models

This section of the literature is the closest to our line of work, as we are approaching the problem by decomposing a terrain into discrete cells that can be freely moved. This specific approach has been tried multiple times before, although with different considerations.

For example, on [23] a similar approach to ours is presented: a rocky object is decomposed into multiple voxels connected by joints, and then a physical simulation is run to determine the forces that each block and joint has to withstand to determine which blocks should be removed. In our case we also use cells and joints to represent the terrain, but our project additionally deals with the effect of water infiltrating the rock and destroying the joints.

Another recent example was presented by [24]. The goal of the paper was to simulate sandstone dynamics with multiple effects such as wind and fluvial erosion. The presented method represents a sandstone structure using a set of particles, and then they modify the model using the following equation for each particle:

$$\frac{\partial b}{\partial t} = -\varepsilon(\sigma_c)(\mathcal{W} + \mathcal{F}) \tag{3.1}$$

Where b represents the *viability* which describes the expectancy that the rock has not been eroded, ε represents the *erodibility*, σ_c is the Cauchy stress tensor, and $\mathcal{W}$ and $\mathcal{F}$ represent the wind and fluvial local erosion effects.

Figure 3.3: Arch produced by wind erosion. Figure taken from [24]

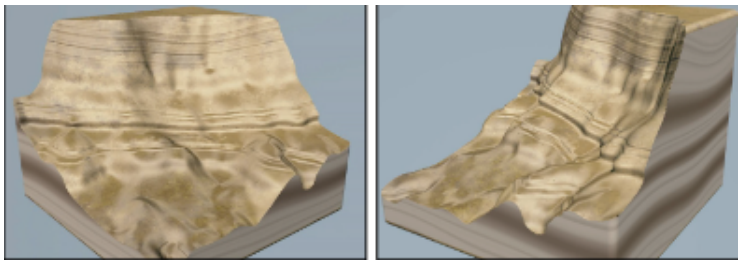


Figure 3.4: Example of erosion using implicit functions, the resulting terrain is smooth due to the nature of implicit functions.Taken from [25]

The problem solved by this simulator is similar to the one that we are tackling, but we are dealing with erosion by freezing and not with wind and fluvial erosion, so our proposed solution will tackle other situations not covered by this method.

Terrain Amplification using Implicit Representations

The previous approaches use some sort of volumetric representation of the model (or at least attempt a volumetric decomposition) to modify individual components of the terrain, but other representations can also be used.

Specifically, the method presented in [25] is based on converting heighfields into implicit surfaces and then working on those implicit surfaces to generate complex 3D features such as arches or caves. To achieve this, they also present a geological model to simulate different natural phenomena such as erosion and produce realistic results.

In order to represent the aforementioned features, the implicit function for the terrain has to be modified, and the resulting terrain will also be given as an implicit function. Implicit functions are really good at capturing smooth features, but representing sharp features such as vertical walls of rocky cliffs is a challenging task. For this reason, this method fails at capturing rough features generated by the erosion process, and this is a limitation that we addressed in our project by using voronoi cells instead of an implicit representation.

Another interesting work in the field is the one presented by [26]. In the article, they present a method to enhance smooth terrains by adding realistic reproductions of rock fractures. The approach taken in the article consists in dividing the original terrain into blocks, whose shape and size is based on the properties of the given rock, and then combining the blocks with some parts of the terrain to obtain a volumetric representation of the fractured bedrock patterns. In this method, the resulting combined model is given

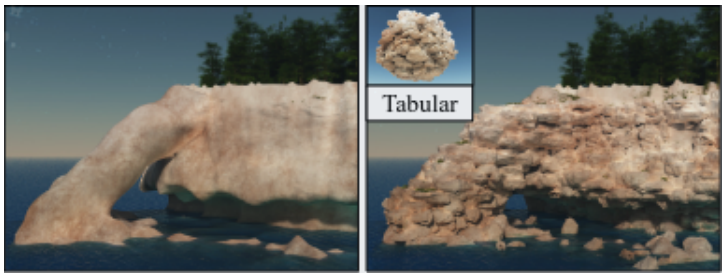


Figure 3.5: Terrain enhanced by generating fractured blocks according to the geology of the terrain. Taken from [26]

as an implicit surface that combines the original terrain with the fractured blocks and, to achieve that, they proposed and applied an operator to reproduce the features of the fractured rock on the implicit surface.

To finish this section on implicit surfaces, the approach presented in [27] has some smilarities with our method. The goal of the paper was to simulate a phenomenon called *Krastik Networks*, which are complex networks of caves, tunnels, and breakout chambers with stalagmites and stalactites that are formed by the dissolution of highly soluble rocks. To simulate the formation of these complex networks, they propose a method that consists in generating tunnel paths that permeate an initially solid terrain and create the different tunnels and caves of the network. Once these paths are constructed, they synthesise a new terrain model using implicit functions. The goal of this article might be different than ours, but our approach also consists in generating paths that permeate the rock to create some erosion effects.

Erosion using Layered Representations

In section 2.2.1 we briefly discussed layered heightfield representations and how they can be used for erosion simulation, specially to simulate sediments and their interactions. Using this strategy, the work of [6] presents an interesting approach that specially focuses on the mutual interaction between vegetation and erosion. In the paper, they divided the landscape into bedrock (unmoving), granular materials that may produce landslides, and vegetation (dead and alive), and with that layered representation, they could run a simulation to modify those layers, representing the effect of erosion in the ecosystem, improving realism and showing the temporal evolution of a landscape. The used layered representation can be appreciated in figure 3.6.

Another approach that uses layered representations is the one presented by [28], which tries to simulate the movement and interaction of tectonic plates to generate mountains with complex folded strata. In the paper, a layered data structure was presented to represent the different tectonic plates and compute the results of their collisions.

In our project, layered representations are not suitable for our goal, as we focus on erosion caused by water entering the pores of the rock and breaking them from inside, causing fractures and potentially separating chunks of rocks from the main structure. Layered representations can represent different features of the landscape with much more memory efficiency than other approaches and can also be used for erosion simulations, but they have limitations that make them not appropriate for certain types of erosion simulation, specially those that consider damage done over all the terrain and not just the interaction of different layers.

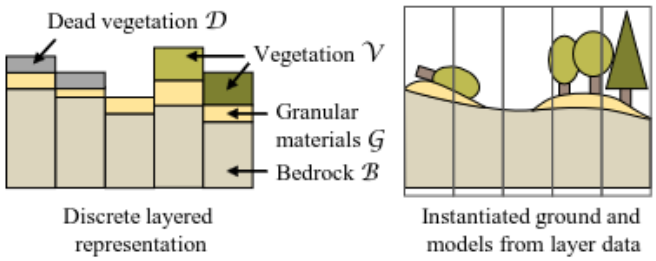


Figure 3.6: Illustration of stacked height functions, used to simulate interactions between vegetation and erosion. figure taken from [6]

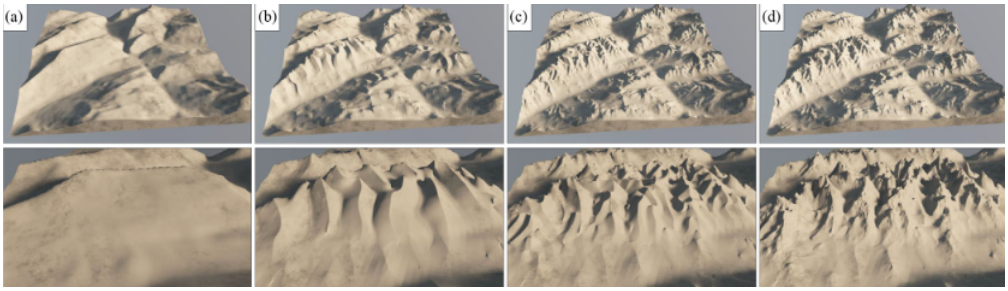


Figure 3.7: Example of terrain enhancenment through adding erosion patterns. Taken from [30]

Machine Learning Approach

In the field of terrain modeling there have also been approaches that use machine and deep learning to improve the realism of the landscape and add complex details. In these types of methods, the goal is usually to add details to the terrain using machine learning models that are trained on real terrains or other suitable synthetic terrains. As they use example landscapes to train the models, erosion features can implicitly (and sometimes explicitly) be generated. These methods are a bit far from our proposed solution, but, despite that, there are interesting techniques using this approach.

In [29] they use a Conditional Generative Adversarial Network trained on real-world terrain to add realistic detail to a rough sketch provided by the user. Moreover, they also have a framework to simulate the evolution of the terrain through erosion (with another adversarial network); [30] presents an approach that procedurally enhances terrains by adding erosion patterns using structured noise, and [31] uses Fully Convolutional Neural Networks to upscale low-resolution terrains into plausible high-resolution ones.

3.3 Base Simulator for this Project

As mentioned in the introduction of this report (1), this thesis is built upon the work of [1], so we started with the algorithm they proposed and made some extensions and improvements. For this reason, in order to understand our project, we first have to understand the method presented in the previous project.

In the original paper [1], they present an approach to simulate terrain erosion that can be divided into two main parts: first, they divide the terrain into independent cells connected by *links* between neighboring pieces, and then, they apply an erosion algorithm to degrade the links and, eventually, remove the parts that have been disconnected from the terrain. Figure 3.8 shows the erosion simulation

in action, with water infiltrating a terrain that has been decomposed into voronoi cells.

To perform the decomposition of the terrain into cells, they sampled multiple points of the model and then generated a voronoi tessellation. In our project we still use the voronoi tessellation to generate the cell graph, but we modified the sampling to obtain better results. The original method used boolean operations between the voronoi tessellation and the original model so that the tessellation would fit the base terrain, but we tried to improve the decomposition so it could better fit the terrain without extra operations, which, in addition to improving the efficiency of the computation, improving the quality of the voronoi cells will also improve the quality of the erosion simulation. The better the decomposition fits the original model, the better the erosion algorithm will fit the terrain conditions.

Regarding the algorithm that simulates water infiltration and link breakage, we implemented a mechanical model to represent link stress and modify those stress values once a link breaks. The extended algorithm now modifies the state of the system after a fracture, instead of just modifying the state of one link, which can produce an effect called crack propagation. The addition of this mechanical model makes the method more based in natural and physical processes, and it improves its realism.

Additionally, we also experimented with multi-resolution decompositions to improve the efficiency of the simulation without compromising too much on its quality. The algorithm can be applied in a less granular decomposition, and then we can modify the more granular one according to the changes on the simplified level.

Furthermore, this thesis also changes many of the tools and the testing framework. The original project implemented a Blender plugin to demonstrate their solution, while we implemented the algorithm into a C++ OpenGL application. Moreover, we have also changed the models we use for our application: In the original method, arbitrary models imported on Blender were used, while we decided to focus on just DEMs, which are a pretty standard representation for terrains in computer graphics.

Finally, even if the basic algorithm is taken from another project, we are going to explain in full detail all parts of the algorithm so that the reader can understand our implementation just by reading this thesis. Keep in mind we took the main ideas of the method and implemented them in our own way, so some smaller details of the implementation may also be different from the original even if we do not highlight the change.

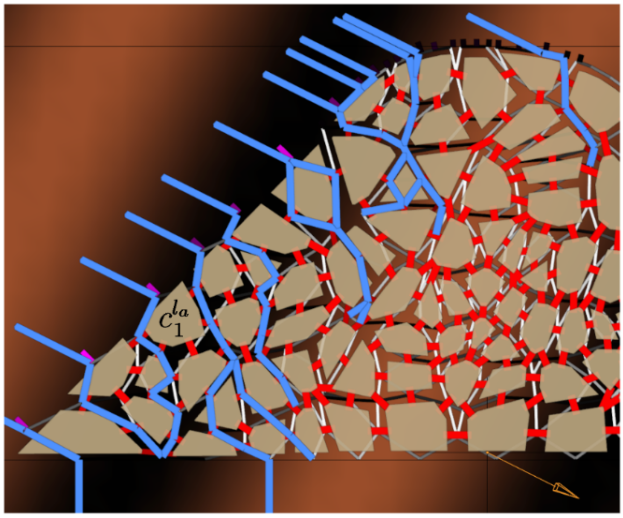


Figure 3.8: Example of water infiltrating a terrain divided into a set of voronoi cells connected by links. Taken from [32]

Chapter 4

Model Decomposition

In order to represent the internal structure of a model, we decomposed the base model into multiple cells covering the whole volume of the object. A core part of this project has been focused on the model decomposition, as well as on algorithms to produce and improve said decomposition. In this section of the thesis we are going to take a deep dive into the basic method taken from [1] and all the improvements we added.

4.1 Base Method Overview

As stated eariler, the main idea of the decomposition process is to divide the original model into multiple shards that are connected with bonds (that we call links). In order to generate the fragments of the models, which will be called cells from now on, a voronoi tessellation is used, as it produces a more natural rocky appearance than other possible decompositions like voxelizations or tetrahedalizations. The process of generating is further divided into multiple steps: **Point Extraction**, **Voronoi Cells Generation** and **Links Generation**. Additionally, we also experimented with multi-resolution decompositions, where one cell on the lower resolution contains multiple cells on the higher resolution level; this adds an additional step that consists in creating the lower resolution levels from the higher one.

In the original project, they divided the decomposition process into different steps, as it can be seen in 4.1. However, due to some changes in the method, we slightly changed said steps. Despite that, the main outline of the process is similar, and our changes reside in the specific steps.

4.2 Point and Cell Generation

Remember that voronoi tessellations are partitions of a model into different regions, where, given a set of centroids $C = \{c_1, c_2, \ldots, c_n\}$, the set of regions $\mathbf{R} = \{R_1, R_2, \ldots, R_n\}$ is defined by:

$$R_i = \{p \in \Omega | \forall c_j \in C, \;\; d(p, c_i) \leq d(p, c_j)\} \tag{4.1}$$

Where $\Omega \in \mathbb{R}^3$ is the domain of our model, d represents the euclidean distance and c_i is the centroid of region i.

This definition implies that the only thing we need in order to construct a voronoi tessellation is a set of centroids, which will define our regions, and then the regions will become our cells defined by the

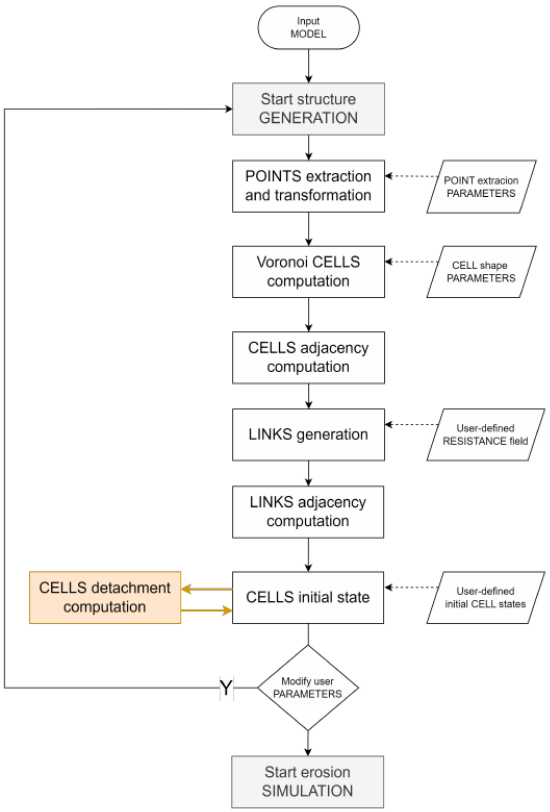


Figure 4.1: Diagram of the original method. Taken from [32]

planes that form their boundaries. The number of centroids chosen and their positions heavily affect the result of the tessellation, so the sampling method that we decide to implement will be highly relevant to the end result. One more thing that we will need to take into account is the fact that the regions that lie on the boundary of the domain will be technically unbounded, and we will have to manually add walls to cut them so they do not leave the boundary of the original model.

4.2.1 Cell Generation

The first step of the voronoi tessellation would be to sample the points that we will use as centroids of our cells. Despite that, we think it is better to first explain the process of constructing the cells from the points, as we took it into account when designing our sampling.

To construct the cells from the centroids, we offload the computation to the library Voro++ [3], which, given a set of points and a boundary (with restrictions on the boundary that will be discussed in the future) gives us a set of polyhedra representing the different voronoi cells. Even if this part of the computation is offloaded to another library, it is important to know about the process of constructing of one voronoi cell.

Given a pair of centroids c_i and c_j, the whole region is divided into two half-spaces: H_{ij}, where points are closer to c_i than c_j; and H_{ji}, for points closer to c_j. More formally, we can define H_{ij} the following way:

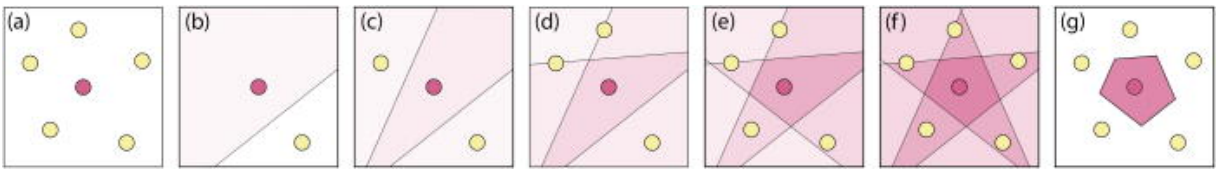


Figure 4.2: Process of construction of a voronoi cell from the intersection of half-spaces. figures (b) to (f) show the process of cutting the space into multiple half-spaces, one half-space at a time. Figure taken from [33]

$$H_{ij} = \{p \in \Omega | d(p, c_i) \leq d(p, c_j)\} \tag{4.2}$$

Every point in R_i is closer to c_i than any other centroid c_k, so it follows that it also has to be in H_{ik} for every $k \neq i$. Conversly, if we have a point $p \in \Omega$ in H_{ik} for every $k \neq i$, it follows that $p \in R_i$. This fact allows us to define the voronoi cell R_i as the intersection of a set of hall-spaces:

$$R_i = \bigcap_{j=1}^{n} H_{ij} \tag{4.3}$$

Each half-space is defined by the plane that is equidistant from both centroids, so this converts the problem of creating a voronoi cell into computing the intersection of multiple planes. Keep in mind that H_{ii} would define all the domain, since every point is at the same distance from c_i and c_i, so it does not have any effect adding it to the intersection. A nice visual demonstration of the construction of a voronoi cell from the intersection of half-spaces is shown in figure 4.2

Before ending the explanation about cells construction, it is important to keep in mind that a pair of centroids define a plane where every point on the plane is equidistant from both centroids. Let us define this plane more formally, as we are going to use it later:

$$P_{ij} = P_{ji} = \{p \in \Omega | d(p, c_i) = d(p, c_j)\} \tag{4.4}$$

Neighboring Information

To run the erosion simulation, we will additionally need to create [links] between neighboring cells, which represent the bonds between different rocks. Adjacency information is also given by the Voro++ [3] library, but we will provide an overview of the method to compute this information.

We have just commented that in order to generate a voronoi cell, you will need to perform an intersection of halfspaces, which would imply incrementally intersecting your partial solution by one plane at a time. Each time you cut your partial solution by a plane P_{ij}, you can store the ID of the particle j that generated said plane. At the end, just a few planes will remain as faces of the voronoi cell, and those planes will have an ID that indicates which other cell is sharing that specific face. Using this method, we will compute the neighboring information while we compute the voronoi cells and we can use this information later to generate the links between cells.

Cell Attributes

To end this section about voronoi cells, it is important to highlight what attributes will be stored for each cell, besides the vertices and edges that define its shape and its adjacency information.

- **Shape Information:** For each cell we will store its vertices (with positions), and its faces (with its vertices).
- **Additional Face Information:** For each face we will also keep its normal, its area, its centroid and the ID of the cell that is on the other side.
- **Cell State:** We define 3 possible states for a cell: **Solid**, which implies the cell belongs on the terrain (and it has not been removed yet by the erosion simulation); **Air**, which implies that the cell does not belong to the model; and **Core**, which makes the cell fixed and imposible to remove. A cell can not have two different states simultaniously.
- **Volume:** For some computations we are also interested in keeping track of the volume of a cell.
- **Boundary Information:** Finally, we will also need to keep track of the solid cells that are on the surface of the model, as they will be in contact with the air and the water infiltration algorithm will start by one of those cells. At the start of the algorithm, exterior cells would be those adjacent to air cells or a wall.

4.2.2 Point Sampling

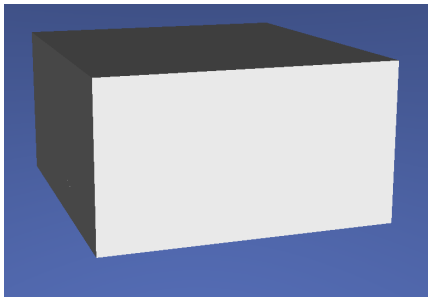
The first part of the method consits in sampling multiple points inside the model to construct a voronoi decomposition. In this thesis, and unlike in the base method presented by [32], we are using DEMs to represent and load our terrains, so we are going to make use of that.

A heightfield $\mathcal{H}$ is basically a 2D grid where each value h_{ij} of the grid; represents the height at one specific cell (i, j), we can then use the data points that we have to obtain the elevation on a continuous domain using interpolation. Because we already have a defined grid, we can use it to implement our sampling. The main idea is to use the 2D grid and extend it to a 3D grid by setting a number of vertical samples (which will be a user-difined parameter) and then sample one point for each element of the grid, as long as the point lies inside the terrain (the 3D grid will represent the bounding box of the terrain, so not every voxel will be inside it).

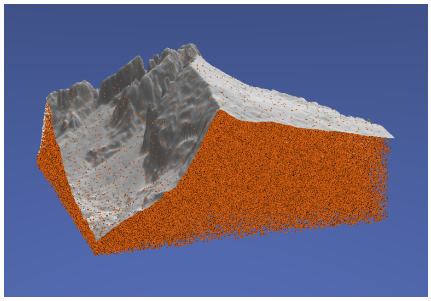
Now that we have a way to uniformly sample points inside the model, we can extend it and slightly break the regularity by shifting each point by a certain quantity. Considering each point to initially be sampled at the center of its grid cell, we are going to shift each point by *strictly* less than half the cell size in a random direction. This way we will obtain a sampling scheme between complete random sampling (white noise) and a completely regular sampling, where we guarantee each point to not be too close or too far from each other. Specifically, from each grid cell center c_{ijk} we create our sample c'_{ijk} using the following equation:

$$c'_{ijk} = c_{ijk} + \begin{bmatrix} r_x s_x \\ r_y s_y \\ r_z s_z \end{bmatrix} \tag{4.5}$$

Where s_x, s_y and s_z are the cell sizes on each direction (we can have rectangular cells) and r_x, r_y and r_z are three different scalars sampled uniformly at random such as $-\frac{1}{2} < -m \leq r_k \leq m < \frac{1}{2}$, where m denotes the maximum ammount of shifting allowed and $k \in \{x, y, z\}$. If m was $\frac{1}{2}$ then samples will be allowed to lie on the boundary of the grid cell, which could result in two samples being at the same place, that is why we set m to be *strictly* smaller than $\frac{1}{2}$.



(a) Voronoi cells of the decomposition



(b) Sample points with the base model as reference

Figure 4.3: Result of the voronoi decomposition without cutting the boundary cells vs. the samples used for the decomposition with the base model as reference

Moreover, we can bound the distance between two arbitrary points in terms of m and the cell sizes s_k. Suppose s_x is the smaller size, then the minimum possible distance between two samples $c_{i,j,k}$ and $c'_{i+1,j,k}$ appears when $c'_{i,j,k} = (x + ms_x, y + \alpha s_y, z + \beta s_z)$ and $c'_{i+1,j,k} = (x + s_x - ms_x, y + \alpha s_y, z + \beta s_z)$, so the distance is:

$$d(c'_{ijk}, c'_{i+1,j,k}) = ||c'_{i+1,jk} - c'_{ijk}|| = ||(s_x - 2ms_x, 0, 0)|| = s_x(1 - 2m) \tag{4.6}$$

Following a similar logic, the maximum distance between $c'_{i,j,k}$ and its closest sample $c'_{i+1,j,k}$, will be when $c'_{i,j,k} = (x - ms_x, y - ms_y, z - ms_z)$ and $c'_{i+1,j,k} = (x + s_x + ms_x, y + ms_y, z + ms_z)$ and the distance would be:

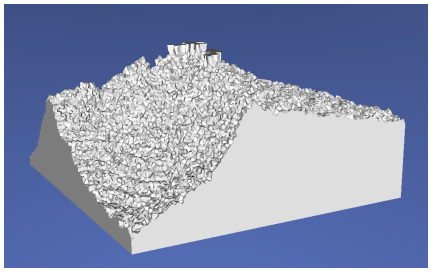
$$d(c'_{ijk}, c'_{i+1,j,k}) = ||c'_{i+1,jk} - c'_{ijk}|| = ||(s_x + 2ms_x, 2ms_y, 2ms_z)|| \tag{4.7}$$

But keep in mind that this maximum distance will only be reached when $i = 0$, as in any other case c'_{ijk} would be closer to $c'_{i-1,j,k}$ than $c'_{i+1,j,k}$ (defining c'_{ijk} and $c_{i+1,j,k}$ as we did). An analogous argument can be repeated with s_y or s_z as the smaller size.

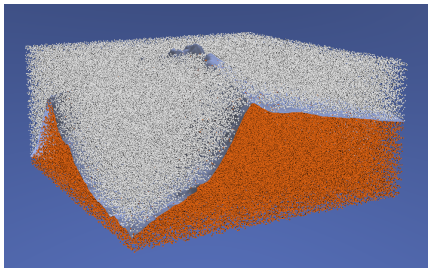
To sum up, the method we have chosen to generate samples bounds the minum and maximum distances between a sample and its closest pair, so it combines some regularity that comes with using a grid with some randomness that breaks the monotony and creates more natural shapes.

4.2.3 Wall Construction

The last consideration we have to make to create the voronoi cells is about dealing with cells on the boundary of the terrain. The method proposed in [32] relied on boolean operations between the model and the set of voronoi cells to cut the boundary cells, but in this project we have tried multiple methods to appropriately create the boundary cells without relying on expensive boolean operations between models. Moreover, the original method only used the boolean operations as a way to showcase the results, but the algorithm relied on cells that were not properly cut, which is another motivation to generate voronoi cells that directly fit the model without additional visual tricks.



(a) Voronoi cells of the decomposition



(b) Sample points with the base model as reference. White particles are out of bounds

Figure 4.4: Result of the voronoi decomposition when allowing sampling out of bounds

Sampling out of Bounds

If we do nothing more than sampling points and computing their voronoi cells, some cells will extend until the bounding box of the terrain, and the resulting decomposition will result in just a giant rectangular prism made of internal voronoi cells. The problem, as we mentioned earlier, is that the boundary voronoi cells are not cut, but we can trivially try to solve this problem by adding more cells outside that boundary, so the cells on the boundary of the terrain stop being the boundary and thus are cut by other cells. The samples outside of the terrain generate *air* cells, so they will not be rendered and the only rendered cells will be the internal ones, which are somehow cut.

The problem of generating a giant rectangular prism is solved using this method, but the surface of the terrain will have a really irregular shape, as cells are arbitrarily cut. Despite that, we can use this idea as a base and then try to improve it so the surface of the model resembles more closely the original.

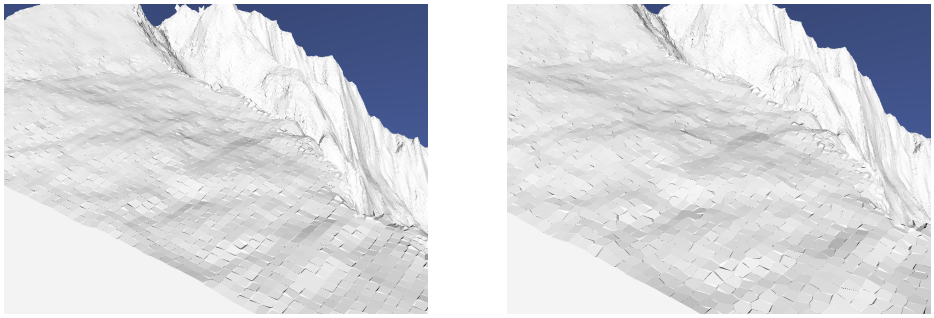
Using Additional Samples to Generate Cutting Planes

As explained in section 4.2.1, every pair of samples defines a plane P_{ij} with all points that are equidistant from both samples. Additionally, we know that region R_i will be cut by all planes P_{ij} such that $i \neq j$. We can use this concept to generate new cutting planes using more samples to send to the voronoi cell generation algorithm.

As mentioned in the preliminaries 2.2.1, we can sample normal vectors from DEMs, which means we can extract a plane from every point on the terrain surface. Using this concept, we can create a plane for every cell, bound it using the size of the cell, and place it on the height of the given cell (so it lies on the surface) to obtain a decomposition of the terrain surface using quads. Moreover, we can use the idea of sampling points outside the boundary to cut the cells on the surface of the terrain to create an approximation of this quad tessellation.

For every grid cell (i, j) of the heightfield $\mathcal{H}$, we can extract its height at the center of the cell h_{ij} and also its normal $n_{i,j}$ to create a plane π_{ij} going through point (x_{ij}, y_{ij}, h_{ij}) with normal n_{ij}. With this, we can create two samples p_{ij} and q_{ij} such that:

$$p_{ij} = \begin{bmatrix} x_{ij} \\ y_{ij} \\ h_{ij} \end{bmatrix} - kn_{ij} \tag{4.8}$$



(a) Results sampling at cell center (b) Results adding some jittering

Figure 4.5: Wall results sampling at the center of every cell vs. adding some jittering

$$q_{ij} = \begin{bmatrix} x_{ij} \\ y_{ij} \\ h_{ij} \end{bmatrix} + kn_{ij} \tag{4.9}$$

So that p_{ij} is inside the terrain, q_{ij} outside and both points are at the same distance k from the plane, which implies that the plane P_{pq} generated by those two points during the voronoi cell computation will be exactly the plane π_{ij}. Applying this concept to every grid cell will produce a better surface than the one generated by just sampling out of bounds, as we cut the boundary cells following the shape of the terrain. The only thing that we should take into account is to choose an appropriate value of k.

Additionally, because we can interpolate the values of the grid to obtain the height $h(x, y)$ and normal $n(x, y)$ for any point inside the grid bounds, we can apply some jittering to break the regularity and create differently shaped cells that still match the surface. Given $x'_{ij} = x_{ij} + r_x s_x$ and $y'_{ij} = y_{ij} + r_y s_y$ (where r and s have the same definition as the one we used in section 4.2.2), we define the shifted points p'_{ij} and q'_{ij} as:

$$p'_{ij} = \begin{bmatrix} x'_{ij} \\ y'_{ij} \\ h(x'_{ij}, y'_{ij}) \end{bmatrix} - kn(x'_{ij}, y'_{ij}) \tag{4.10}$$

$$q'_{ij} = \begin{bmatrix} x'_{ij} \\ y'_{ij} \\ h(x'_{ij}, y'_{ij}) \end{bmatrix} + kn(x'_{ij}, y'_{ij}) \tag{4.11}$$

The idea of this wall generation method is to mix it with the general sampling method that we described, so we will have all samples from the general method and two additional samples per cell that will represent the surface. This mix of samples initially supposes a problem, as the minimum distance bound that we showed in section 4.2.2 might not hold between points sampled from different methods. When we tried to mix both sampling methods without further consideration, some bumps appeared (as seen in figure 4.6) on the surface of the terrain due to a mix of samples close to the surface. To solve this, we limited the allowed height for general samples so that the only samples close to the surface were the ones that corresponded to the walls.

Finally, there was another problem when using this method to create the walls. When we tried to execute the method as described, we obtained great results on relatively flat surfaces, but for sharp peaks

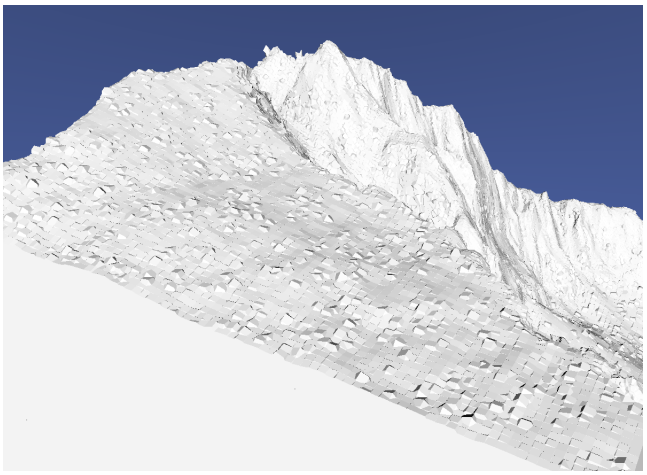


Figure 4.6: Bumps appearing due to an unappropriate mix of the two sampling methods

and places with a lot of inclination, we obtained some sharp long faces that extended beyond the limits of the terrain. An example of this problem is shown in figure 4.7 along with a comparison to the original model.

After encountering these unexpected results, we hypothesized that these artifacts could be related to the *Area Ratio* of the terrain. Consider a 2D square box Ω with extremes (x_{min}, y_{min}) and (x_{max}, y_{max}), then consider the portion of the terrain $\mathcal{T}$ that is defined by the elevation function for $(x, y) \in \Omega$:

$$\mathcal{T}(x, y) = \begin{bmatrix} x \\ y \\ h(x, y) \end{bmatrix} \tag{4.12}$$

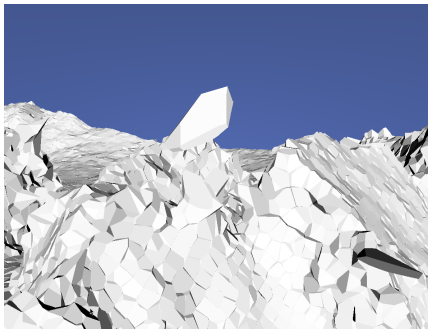
With this, we can measure the ratio between the area of the box A_Ω and the surface $A_\mathcal{T}$. Right now we are taking one sample for each grid cell in order to construct a wall, which implies that every equally sized subsection of the grid Ω will have the same number of samples, and this leads to the following problem: A subsection Ω with $\frac{A_\mathcal{T}}{A_\Omega} = 2$ will use the same samples as another section Ω' with $\frac{A_{\mathcal{T}'}}{A_{\Omega'}} = 1$ but it will be covering twice the area of the surface. In [34], this concept of the ratio between areas is defined for every point on a terrain with the name of *rugosity*:

$$\rho(x, y) = \frac{A_\mathcal{T}(x, y)}{A_\Omega(x, y)} \tag{4.13}$$

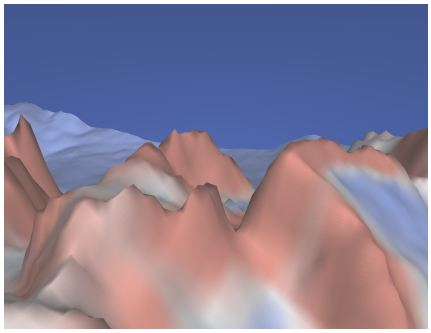
Where Ω is defined by a squared window centered at (x, y).

If our problem is caused by using too few samples to cover some portions of the surface with a lot of area, we can decide to increase the sampling rate on parts of the surface with high rugosity. Thanks to our continuous normal and elevation surfaces, we can easily divide a grid cell into four sub-cells, sampling the elevation and normal at the centroid of each sub-cell (with optional jittering inside the sub-cell), and then create a pair of samples for each one like we were doing before. We can even go further than this and subdivide each sub-cell into four pieces more. This way we could take $2 \cdot 4^k$ samples per cell for $k \in \{0, 1, \ldots\}$ depending on the rugosity: lower amounts of rugosity would imply lower values of k. In our case, we have obtained good results using $k \in \{0, 1, 2\}$, so we have not tested further subdivisions.

As a final note, it is important to consider that the higher the elevation variation (imagine a surface

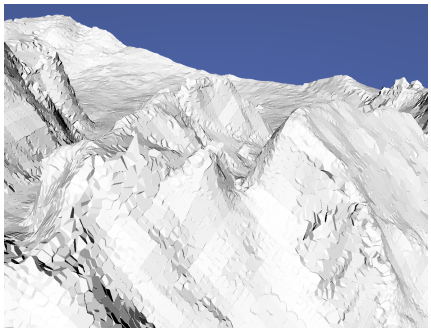


(a) Base results of our wall generating method

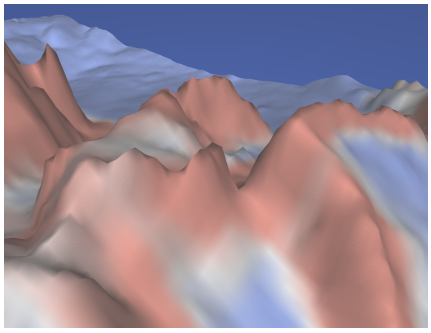


(b) Base terrain painted with a Cool-Warm color gradient. The warmer the color the higher the rugosity of that point

Figure 4.7: Results of the wall generation process vs. a color map over the base terrain showing the rugosity at each point



(a) Results of our wall generating method after appliying subsampling based on rugosity



(b) Base terrain painted with a Cool-Warm color gradient. The warmer the color the higher the rugosity of that point

Figure 4.8: Results of the wall generation process vs. a color map over the base terrain showing the rugosity at each point

going up and down multiple times over a small number of cells), the higher would be its rugosity, so the subsampling method we implemented will cover us from those cases as well as cases where we just have a really high slope.

4.3 Link Generation

After generating all voronoi cells and computing its neighboring information, we are prepared to create the links between the cells, which will represent different properties of the bonds between the rocks.

4.3.1 Link Parameters

In order to properly describe link generation, we first have to understand which properties we will need to define for each link:

- **Link State:** A link can be in one of three states: **Exterior**, **Interior** and **Broken**. Exterior links are those that connect an exterior cell with an air cell.

- **Link Life:** The erosion simulation will damage the bonds between the rocks, this damage is represented by the "life" parameter.
- **Normal and Shear Stresses:** For our erosion simulation we will also need to keep track of the stresses that each link has to withstand. We will explain how to compute the stresses in a future section.
- **Neighbors:** The hardest thing to compute, but one of the most important, would be the adjacency information between links. For each link we need to know which links water can jump to.

4.3.2 Link Adjacency Information

Our erosion simulation method requires a graph between the links to represent water movement. Water moves through the cracks between rocks, which means that the water will move from one face of a cell to another, and we cannot just use the general adjacency information of the cells to represent these movements.

On a 3D model formed by faces, vertices, and edges, we say that two faces are adjacent if and only if they share an edge. Conceptually, if two faces share an edge, then you can move from one face to the other by crossing the common edge. As there will be a link for each face of a cell, the adjacency information of the faces will also tell us the adjacency information of the links: Given the neighbors of each face of every cell, the neighbors of a given link can be computed as the union of the neighbors of the two faces it connects (technically a link only connects two cells that share one single face, but we can split the face in two, as it will appear in two different cells with different adjacency information).

More formally, given face f of cell i, defining the set of edges of cell i as E_i and the set of faces as F_i, then the set of neighbors of f is defined as:

$$\mathcal{N}(f) = \{f' \in F_i | \exists e \in E_i,\ e \in f' \text{ and } e \in f\} \tag{4.14}$$

And with the definition of $\mathcal{N}(f)$ we can define the neighbors of a link l that connects faces f_1 and f_2 as:

$$\mathcal{N}(l) = \mathcal{N}(f_1) \cup \mathcal{N}(f_2) \tag{4.15}$$

And with this we can create a graph connecting all links of our decomposition, which is the last thing we needed as preparation for the erosion simulation algorithm.

4.4 Multi-Resolution Extension

With all we explained up to this point, we have enough information to run the erosion simulation. Still, the algorithm can be quite heavy for big terrains with hundreds of thousands of cells, or even millions. For that reason, we present a method to speed up the erosion computation without compromising on the quality of the decomposition.

The main idea is to compute everything normally as we mentioned (sample points, generate the cells, and then create the links), and once the decomposition is created, we then select a subset of all samples, construct another voronoi tessellation from that subset, and store for every original sample the closest

point on the subsampled set. This will result in two resolutions of voronoi tessellations that are linked, so we can execute the erosion simulation on the lower resolution decomposition and then modify the higher resolution one according to the produced changes. The quality of the terrain that the user sees will be the same, and the erosion simulation will be faster, although we will be sacrificing some quality in the erosion process.

4.4.1 Furthest Point Subsampling

If the goal is to obtain a subset of the original N samples to represent a lower-resolution version of the terrain, the easiest thing we can do is pick at random N/s elements of the original set, where $1/s$ is the fraction of samples we want to pick; however, choosing points at random leads to many problems. For example, we could have some regions packed with subsamples, each representing a small portion of the terrain, and some regions with very few subsamples, producing the opposite problem.

Random subsampling is highly irregular, so we need a better sampling method to guarantee some amount of regularity and that every subsample represents a roughly equal region of the terrain. To achieve this goal, we have chosen the *Furthest Point Sampling* method. Given a set of points $P = \{p_1, \ldots, p_N\}$ we want to create a subset S of size $|S| = N/s$. In order to achieve this, we first pick one element of P at random to be the first one of S; then, we compute the distance of every point $p_i \in P$ to the set S and add the furthest point to S. We repeat this process iteratively until we have added N/s samples to S. Algorithm 1 shows the pseudocode of the method we used to obtain a subsample.

Algorithm 1 Furthest Point Sampling

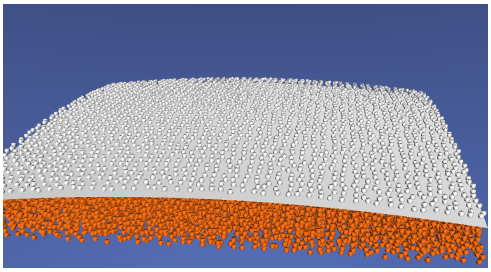
$p_r \leftarrow$ random sample from P
$S \leftarrow S \cup \{p_r\}$
while $|S| < N/s$ **do**
  $p_k \leftarrow \arg\max_{p_i \in P} d(p_i, S)$
  $S \leftarrow S \cup \{p_k\}$
end while

We define the distance between a point and a set, using the minimum euclidean distance:

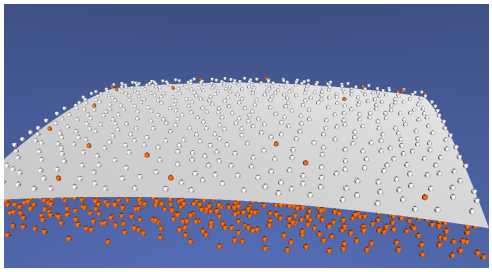
$$d(p_i, S) = \min_{s_i \in S}(d(p_i, s_i)) \tag{4.16}$$

After obtaining subset S, the only thing left to do is linking the points of P to the closest subsample of S. This way, we could perform our erosion algorithm with only the cells on S, and when a cell s_i of the subsample is removed, we can remove all cells of P that are linked to s_i.

Before ending this section, there is a final consideration that we have to take into account. The original group of points P contains points marked as solid (inside the terrain) as well as points marked as air (outside the terrain), so the subsample S may also contain points outside the terrain. If we mark the subsampled points as air when they are outside of the terrain, a huge problem arises: Because we will never remove air cells in the erosion algorithm, a solid point of P linked with an air point of S will never be removed. To solve this, we will mark as solid all subsamples from S that are linked to at least one solid point on P. This ends up in cells clearly outside the terrain limits marked as solid, which will produce some artifacts in the low-resolution version. Figure 4.9 shows how some solid cells end up outside of the terrain using this method.

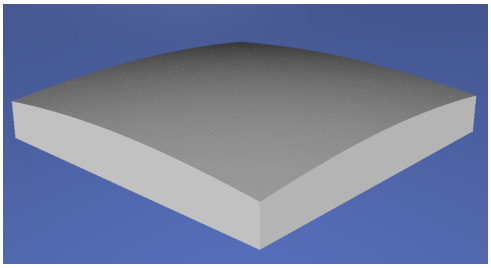


(a) Base sampling along the original surface of the terrain separating air and solid cells

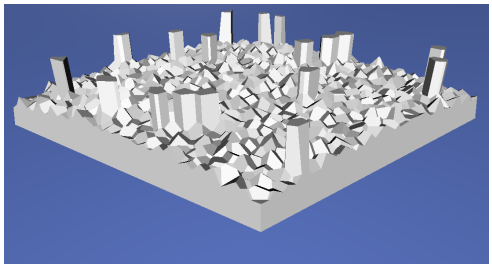


(b) Subsampling using furthest point, some points corresponding to solid cells lie outside of the terrain

Figure 4.9: Results of the furthest point sampling. Some points outside of the terrain are marked as solid because they are linked with solid cells of the original sampling



(a) Decomposition results using the base sampling



(b) Low-resolution decomposition using furthest point to collect a subsample

Figure 4.10: Results of the furthest point sampling. Some artifacts arise due to cells outside of the terrain being marked as solid

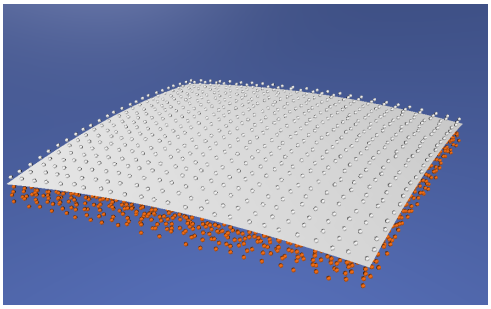
4.4.2 Subsampling Improvement

The current subsampling method will produce some artifacts in the resulting decomposition due to some cells being marked as solid when they are completely outside the terrain (the artifacts can be appreciated in figure 4.10). The low-resolution terrain is not intended to be seen, just to be used to simplify the erosion algorithm, so the visual quality is not highly relevant for our needs. Despite this, even if we do not care about the visual quality, having a decomposition that properly fits the terrain is also important to produce better results during the simulation. For this reason, we dedicated some time to improve this subsampling.

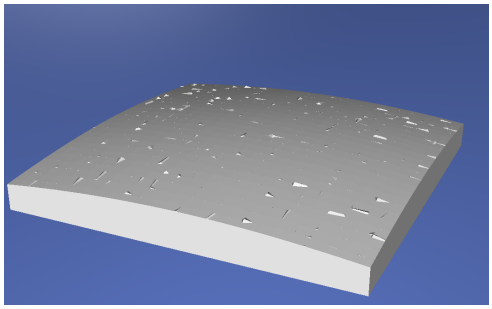
Our current method has the issue of cells outside of the terrain being classified as solid, but it also has another problem: because we are not using anything to explicitly cut the cells at the surface, the results will be similar to the ones obtained when drawing random samples outside of the terrain (figure 4.4). We can solve both problems at the same time by reusing the sampling method we designed for the high-resolution decomposition.

The proposed base sampling method is actually a combination of two methods: one to obtain samples inside the terrain representing the interior, and another that obtains samples to reproduce the surface. Using this information, we can also produce the subsampling in two rounds: one subsampling for the internal sampling and another for the surface.

To obtain the internal samples, we divide all space into a 3D grid and we take one sample per cell



(a) Subsampling using the improved method. Notice how there are no solid cells outside of the terrain



(b) Low-resolution decomposition using the improved method. The walls are not perfect, but they approximate the shape of the terrain, unlike the results generated using furthest point sampling

Figure 4.11: Results of the improved method. The left image shows the subsamples obtained, while the right one shows the final result

of the grid. We can reuse the 3D grid used for the base sampling and create a low-resolution 3D grid made of blocks of cells (for example, we can group a block of 2x2x2 cells into a bigger cell). Using this low-resolution 3D grid, we classify all points in the original set P depending on which block of cells they fall into. Next, for each block of samples P_{ijk} we will choose a *representative* to add it to our subsampled set S. Defining q_{ijk} as the centroid of the block of cells, our representative s_{ijk} will be chosen using:

$$s_{ijk} = \arg \min_{p \in P_{ijk}} d(p, q_{ijk}) \tag{4.17}$$

That is, we will choose the colsest sample to the centroid as a representative.

For the second phase of the sampling, the one that generates the walls, we are using the 2D grid of the DEM to generate pairs of samples at the surface of the terrain. We can use the same idea as with the 3D grid and divide the 2D grid into blocks, choosing a pair of samples per block of cells. Basically, we will create two groups P_{ij} and Q_{ij} for each block and choose a representative of each of those groups. The representative of group P_{ij} will be marked as solid, and the one of group Q_{ij} will be air. P_{ij} will only contain the members of the pairs that belong to the inside of the terrain, and Q_{ij} only the members that lie outside, so we solve the problem of *hybrid blocks* (which contain both air and solid samples) using this method. Additionally, because the members of P_{ij} and Q_{ij} are sampled in pairs to create a wall representing the surface of the terrain, the representatives of each group will also create a wall (although not as precise) that roughly represents the surface, solving our second problem.

Chapter 5

Erosion

In this chapter, we are going to look into the algorithm that we implemented to simulate the erosion of rocky terrains. First, we are going to review the original method that we are extending, and then we are going to explain the mechanical model we implemented as an extension for the algorithm.

5.1 Base Method Overview

The algorithm described in [1] consists in simulating water infiltrating the cracks on the rocks, which are described by the links between cells that we described how to create in the previous section.

The water infiltration simulation is performed by generating different paths using a stochastic flow method. At the start of each path, one exterior link (described in 4.3.1) is chosen, and then one of its neighbors is selected to jump to. Then, the process of choosing a neighbor is repeated iteratively until a stop condition is met. Every time the path crosses a link, the amount of water in the path is reduced and the link is damaged according to the amount of water absorbed. When the damage exceeds a threshold (when the life of the link reaches 0), then the link is broken and the adjacency information of the cells is updated. When a broken link separates the cell graph into multiple connected components, then the smaller components are deleted, unless they contain core cells, in which case the components that do not contain core cells are deleted.

In addition to this base algorithm, we propose a mechanical model to more accurately measure the stress that each link has to withstand to maintain the equilibrium of the system. This proposed model will change the method used to determine when a link is broken and also create a *crack propagation* effect: once a link is broken, the stresses being applied on each link change (in order to maintain the new equilibrium), and this could cause more links to break due to not being able to withstand the new stresses.

The criterion used to choose where the water jumps to next (or where it starts), and the parameters that control how much water is absorbed or when a link is considered broken will be explained after we describe the mechanical model we implemented, as it has an influence over these parameters that control the water infiltration process.

5.2 Mechanical Model

In this thesis we designed a mechanical method to simulate the terrain situation and potentially modify it when link stresses go over a limit. The model we present is based on the *Bonded Discrete Element Method*, which considers the terrain as a set of discrete elements connected by bonds that can move and rotate independently.

In our simulation, the terrain is at equilibrium with gravity as the only force external force. Because the system is at equilibrium, the sum of forces on all links, represented by virtual springs, must compensate for the gravity pull, which implies that some cells might perceive tiny displacements (in order to produce the forces on the links). Based on the cell displacements, we can obtain the stress that each link withstands, and this stress can be used both for the water infiltration algorithm and also to perform crack propagation.

5.2.1 Rotation Preliminaries and Notation

Before we start describing our model, it is important to clearly explain what do we use to describe rotations, as we are making some assumptions that significantly change the way we deal with them.

First, we are going to allow the cells to rotate, so every cell will have a potential angular displacement ϕ along the $(\hat{\theta_1}, \hat{\theta_2}, \hat{\theta_3})$ axis. Angular displacements cannot be considered as vectors, as the addition between two displacements is not commutative, but despite that we are going to abuse the notation and represent them as a vector θ with magnitude ϕ and direction $(\hat{\theta_1}, \hat{\theta_2}, \hat{\theta_3})$. In this thesis we are not going to add two different angular displacement vectors (or not directly), so we are not going to have problems with non-commutativity.

Additionally, the cells are supposed to have extremely small displacements and rotations, so we can approximate angular displacements as infinitesimal rotations. Infinitesimal rotation vectors describe an infinitesimally small rotation and have some interesting properties. Given an infinitesimal rotation vector θ, we can define an infinitesimal rotation matrix Θ as follows:

$$\Theta = \begin{bmatrix} 0 & -\theta_3 & \theta_2 \\ \theta_3 & 0 & -\theta_1 \\ -\theta_2 & \theta_1 & 0 \end{bmatrix} \tag{5.1}$$

Θ is a skew-symmetric matrix and it has the following property for any vector $v \in \mathbb{R}^3$:

$$\Theta v = \theta \times v \tag{5.2}$$

So we can convert any rotation into a cross product. This treatment of rotations as infinitesimal rotations is really useful to us as we are getting rid of the non-linear terms (*sin* and *cos*) of the usual rotation matrices so we can describe rotations on linear equations.

5.2.2 Basic Considerations

In order to properly define our model, we have to start by stating the different assumptions and considerations that we make. First, we assume that we have N independent cells that are connected by some links. Defining F_{ij} as the force applied on the link that goes from cell i to cell j, we can define the total force that is applied to a cell i from its links as:

$$F_i = \sum_{j \in \mathcal{N}(i)} F_{ij} \tag{5.3}$$

Where $\mathcal{N}(i)$ defines the set of neighbors of cell i. Because the whole system is at rest, and the only external force that is applied to it is gravity $\vec{g}$, we know that the following must hold:

$$\sum_{i=1}^{N} F_i = -m_i \vec{g} \tag{5.4}$$

Using cell mass m_i. We assume that the cells can move and also rotate, so we need to explicitly state that the total torque, also known as moment of force, applied to a cell has to be null. Similarly to the forces, the torques are applied to a cell via its links, so, defining M_{ij} as the moment of force applied on the link that goes from i to j, then we can define M_i as:

$$M_i = \sum_{j \in \mathcal{N}(i)} M_{ij} \tag{5.5}$$

And if we consider the link to be placed on the centroid c_{ij} of the interface between cells i and j, which have centroids c_i and c_j, then we can define M_{ij} as:

$$M_{ij} = (c_{ij} - c_i) \times F_{ij} \tag{5.6}$$

And given that the system is at rest we also have:

$$\sum_{i=1}^{N} M_i = 0 \tag{5.7}$$

Now, remember that we can move and rotate each cell, so we will also have to keep track of, for each cell, its displacement s and its angular displacement θ. [1]. As we are simulating the links as virtual springs, these displacements and rotations have a direct influence over the forces of each link. Given the displacements s_i, s_j and rotations θ_i, θ_j of cells i and j, the displacements d_{ij} and d_{ji} of face centroids c_{ij} and c_{ji} will be:

$$d_{ij} = s_i + \theta_i \times (c_{ij} - c_i) \tag{5.8}$$

$$d_{ji} = s_j + \theta_j \times (c_{ji} - c_j) \tag{5.9}$$

Once we have the face centroid's displacements we can compute the forces F_{ij} and F_{ji} applied to each link:

$$F_{ij} = k_t(d_{ij} - d_{ji}) \tag{5.10}$$

$$F_{ji} = k_t(d_{ji} - d_{ij}) = -F_{ij} \tag{5.11}$$

Where k_t represents the *total* stiffness of the string, and its exact definition will be provided later. Finally, up until now we have been separating each link (i, j) into two links (i, j) and (j, i) but we only have one link for each interface, which will suffer a single stress (or rather one single shear stress and

[1] θ is properly defined in section 5.2.1

one single normal stress), for that reason we are only going to consider links (i, j) where $i < j$ when performing stress measurements and computing link damage.

5.2.3 Cell Movement

After reviewing some basics, we are ready to define how we are going to obtain cell displacements so we can compute the forces that are applied on each link. First, we are going to define a displacement and roattion vector u_i for each cell i:

$$u_i = \begin{bmatrix} s_i \\ \theta_i \end{bmatrix} \tag{5.12}$$

Where s_i and θ_i are 3x1 vectors, so u_i is a 6x1 vector. Now, in order to introduce the method, let us first consider a simple situation with only two cells i and j. Because we only have 2 cells, $F_{ij} = F_i$, $F_{ji} = F_j$, $M_{ij} = M_i$, and $M_{ji} = M_j$ and we can attempt to describe these values as a system of equations as follows:

$$K \begin{bmatrix} s_i \\ \theta_i \\ s_j \\ \theta_j \end{bmatrix} = \begin{bmatrix} F_i \\ M_i \\ F_j \\ M_j \end{bmatrix} \tag{5.13}$$

Where we just have to define the 12x12 K matrix to make the equation correct. In order to define K, we are going to use the total stiffness k_t (which will be properly defined later), the vectors $r_i = (c_{ij} - c_i)$ and $r_j = (c_{ji} - c_j)$ and a *skew-symmetric* operator $[\cdot]_\times$ defined as:

$$[w]_\times = \begin{bmatrix} 0 & -w_3 & w_2 \\ w_3 & 0 & -w_1 \\ -w_2 & w_1 & 0 \end{bmatrix} \tag{5.14}$$

Which has the same structure as the infinitesimal rotation matrix described in section 5.2.1, also keeping the property of $[w]_\times v = w \times v$. Additionally, note that the generated matrix has the property $[w]_\times^T = -[w]_\times$. With this, we can define K as:

$$K = \begin{bmatrix} k_t & k_t[r_i]_\times^T & -k_t & -k_t[r_j]_\times^T \\ [r_i]_\times k_t & [r_i]_\times k_t [r_i]_\times^T & -[r_i]_\times k_t & -[r_i]_\times k_t [r_j]_\times^T \\ -k_t & -k_t[r_i]_\times^T & k_t & k_t[r_j]_\times^T \\ -[r_j]_\times k_t & -[r_j]_\times k_t [r_i]_\times^T & [r_j]_\times k_t & [r_j]_\times k_t [r_j]_\times^T \end{bmatrix} \tag{5.15}$$

Where every element is a 3x3 matrix, so K is a 12x12 matrix. If we apply this matrix to the displacements and rotations we get:

$$\begin{bmatrix} k_t & k_t[r_i]_\times^T & -k_t & -k_t[r_j]_\times^T \\ [r_i]_\times k_t & [r_i]_\times k_t [r_i]_\times^T & -[r_i]_\times k_t & -[r_i]_\times k_t [r_j]_\times^T \\ -k_t & -k_t[r_i]_\times^T & k_t & k_t[r_j]_\times^T \\ -[r_j]_\times k_t & -[r_j]_\times k_t [r_i]_\times^T & [r_j]_\times k_t & [r_j]_\times k_t [r_j]_\times^T \end{bmatrix} \begin{bmatrix} s_i \\ \theta_i \\ s_j \\ \theta_j \end{bmatrix} = \begin{bmatrix} F_i \\ M_i \\ F_j \\ M_j \end{bmatrix} \tag{5.16}$$

If we perform the matrix multiplication to obtain F_i, F_j, M_i and M_j we then get:

$$\begin{aligned} F_i &= k_t(s_i + [r_i]_\times^T \theta_i - s_j - [r_j]_\times^T \theta_j) \\ &= k_t(s_i + \theta_i \times (c_{ij} - c_i) - (s_j + \theta_j \times (c_{ji} - c_j))) \\ &= k_t(d_{ij} - d_{ji}) = F_{ij} \end{aligned} \tag{5.17}$$

$$\begin{aligned} M_i &= [r_i]_\times k_t(s_i + [r_i]_\times^T \theta_i - s_j - [r_j]_\times^T \theta_j) \\ &= (c_{ij} - c_i) \times F_i \\ &= (c_{ij} - c_i) \times F_{ij} = M_{ij} \end{aligned} \tag{5.18}$$

$$\begin{aligned} F_j &= k_t(s_j + [r_j]_\times^T \theta_j - s_i - [r_i]_\times^T \theta_i) \\ &= k_t(s_j + \theta_j \times (c_{ji} - c_j) - (s_i + \theta_i \times (c_{ij} - c_i))) \\ &= k_t(d_{ji} - d_{ij}) = F_{ji} \end{aligned} \tag{5.19}$$

$$\begin{aligned} M_j &= [r_j]_\times k_t(s_j + [r_j]_\times^T \theta_j - s_i - [r_i]_\times^T \theta_i) \\ &= (c_{ji} - c_j) \times F_j \\ &= (c_{ji} - c_j) \times F_{ji} = M_{ji} \end{aligned} \tag{5.20}$$

So we can confirm that the matrix K is defined properly. We can write this same multiplication using a more compact notation, grouping the force and moment of a cell in a single vector $\mathbf{F}$:

$$\begin{bmatrix} K_{ii} & K_{ij} \\ K_{ji} & K_{jj} \end{bmatrix} \begin{bmatrix} u_i \\ u_j \end{bmatrix} = \begin{bmatrix} \mathbf{F}_i \\ \mathbf{F}_j \end{bmatrix} \tag{5.21}$$

And using this compact notation we are ready to define a full system of equations describing all the terrain:

$$\begin{bmatrix} K_{1,1} & \dots & K_{1,N} \\ \vdots & \ddots & \vdots \\ K_{N,1} & \dots & K_{N,N} \end{bmatrix} \begin{bmatrix} u_1 \\ \vdots \\ u_N \end{bmatrix} = \begin{bmatrix} \mathbf{F}_1 \\ \vdots \\ \mathbf{F}_N \end{bmatrix} \tag{5.22}$$

Where the non-diagonal entries K_{ij} will describe the effect of moving cell j on $\mathbf{F_i}$ and the diagonal entries K_{ii} will describe the effect of moving cell i in the resulting force. Keep in mind that, as we are considering multiple interfaces, the diagonal entries will contain a sum of contributions, one for each interface. Using any linear algebra solver (in our case, we are going to use the Eigen c++ library [35]), we can obtain all displacements $\mathbf{u}$ to then compute the different link stresses.

To end this explanation of the system, we already stated that the only force being applied on the system is gravity, and the torques on each cell are also null, so $\mathbf{F}_i$ is going to be defined as:

$$\mathbf{F}_i = \begin{bmatrix} F_i \\ M_i \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ -m_i g \\ 0 \\ 0 \\ 0 \end{bmatrix} \tag{5.23}$$

Finally, there is one more thing to consider when solving and constructing the system. The terrain can translate and rotate in infiniteley many ways with the constraints we have set. For that reason, we will have to add some *fixed* cells that will constrain our problem even more so we can find a concrete solution. For a fixed cell k, we will set $\mathbf{F}_k = 0_{6,1}$, $K_{k,k} = I_{6,6}$ and for every other cell j, $K_{k,j} = K_{j,k} = 0_{6,6}$. Our fixed cells will be the cells with state *Core*, which will be selected by the user.

5.2.4 Stiffness Matrices

We used multiple times the total stiffness matrix k_t without explaining how it is computed. If we consider a spr0ing with normal stiffness k_n and shear stiffness k_s (which are scalar values representing stiffness per unit area), then the force generated by a displacement s of the spring is given as:

$$F = k_n A(s \cdot \hat{n})\hat{n} + k_s A[1 - (s \cdot \hat{n})\hat{n}] \tag{5.24}$$

Where $\hat{n}$ is the unit vector pointing in the direction of the spring, which in our case will be the normal vector of the cell faces, and A is its surface area. The equation basically decomposes the displacement into displacement in the normal direction (that will be affected by the normal stiffness) and displacement in any other direction (affected by the shear stiffness). We need to isolate the displacements in order to solve our system, so we can reorganize equation 5.24 into:

$$F = A(k_n[\hat{n} \otimes \hat{n}] + k_s[I - (\hat{n} \otimes \hat{n})])s \tag{5.25}$$

Where $\otimes$ is the outer product [2] and I the identity matrix. This reorganization allows us to define k_t as:

$$k_t = A[k_n(\hat{n} \otimes \hat{n}) + k_s(I - \hat{n} \otimes \hat{n})] \tag{5.26}$$

Finally, k_n and k_s describe the stiffness per unit area and they are defined in terms of the *Young's Modulus* E and *Poisson's Ratio* υ which are material properties. More Specifically:

$$k_n = \frac{E}{L} \tag{5.27}$$

$$k_s = \frac{E}{2(1+\upsilon)L} \tag{5.28}$$

Where L represents a length. The force applied to a link is proportional to its displacement, so if we used a fixed stiffness that only depended on E and υ, we would have huge variations of results depending on the scale we used to represent the model. For that reason, we are going to use the average of the three sizes of the grid s_x, s_y and s_z to define $L = \frac{s_x+s_y+s_z}{3}$. Additionally, we can also look at the dimensions

[2] The outer product of two vectors $u \otimes v$ is equivalent to uv^T, in case the reader is more familiar with that notation.

of E (Which represents pressure) to obtain that it has to be divided by a lenght so that everything fits (remember that stiffness is then multiplied by area and lenght to obtain a force).

5.2.5 Link Damage Model

Once we computed the stiffness matrix K and solved the system of equations 5.22, the only think left to do is computing the link stresses and determine if the link breaks as a consequence of the stress it has to withstand.

Link Stresses and Failure

Using link force F_{ij} defined in equation 5.10, and only the force F_{ij} to avoid computing duplicate stresses, we define the normal stress σ_n and the shear stress τ as:

$$\sigma_n = \frac{F_{ij} \cdot \hat{n}_{ij}}{A_{ij}} \tag{5.29}$$

$$\tau = \frac{||F_{ij} - (F_{ij} \cdot \hat{n}_{ij})\hat{n}_{ij}||}{A_{ij}} \tag{5.30}$$

$\sigma_n > 0$ defines tensile stress and $\sigma_n < 0$ defines compression. The link (i, j) will break when either of the three failure conditions is met.

First, the *Ultimate Tensile Strenght* T_{max} defines the maximum tension a material can withstand, so the tensile failure condition will be defined as:

$$\sigma_n > T_{max} \tag{5.31}$$

Then, for compression, the *Uniaxial Compressive Strength* UCS of a material defines the compressive limit:

$$\sigma_n < -UCS \tag{5.32}$$

Finally, the *Mohr-Coulomb Criterion* [36], widely used in fracture and material analysis, defines the maximum shear stress a material can withstand. Compression makes shear failure harder, and tension makes it easier, so modeling shear failure is more complex than modeling tensile or compressive failure. Still, using the cohesion c of a material and the coefficient of internal friction $tan(\phi)$ we can represent the third failure condition as:

$$\tau > c - \sigma_n tan(\phi) \tag{5.33}$$

Once a link (i, j) has been broken, force F_{ij} disappears from the system, breaking the equilibrium along the link. For that reason, once a link breaks, we will have to recompute the new equilibrium, update the stresses, and check again if any link surpasses the stress limit, and recursively repeat the process in case any link breaks. The most physically accurate way to approach crack propagation would be to break only the link that surpasses the stress thresholds by the most (representing the one that would break first) and then recompute the system until all links are within the margins. However, in order to speed up the computation, we will be removing all links above the threshold each time we recompute the system.

Link Damage

Now we can technically run our algorithm and possibly have some links breaking, but at no point of the mechanical model are we considering the effect of water infiltrating the rock and damaging the links. For this reason we will have a damage variable D for every link, which will go from 0 to 1 and it will be the complement of the life of the link: $D = 1 - life$. Using variable D, we will modify the variables T_{max}, UCS and c that control the failure conditions, so a damaged link will be less resistant to stress, and it will break more easily. The criterion that we use to modify the failure parameters is a simple interpolation using the baseline parameters T_0, UCS_0 and c_0:

$$T_{max}(D) = T_0(1 - D) \tag{5.34}$$

$$c(D) = c_0(1 - D) \tag{5.35}$$

$$UCS(D) = UCS_0(1 - D) \tag{5.36}$$

Notice how, when $D = 1$, then any amount of stress would break the link, so the link will be automatically broken if it reaches this point. The method we use to modify the damage variable of a link depends on the water absorbed by said link, and because of that dependency, the specific damage equations will be described in section 5.3.2, after an explanation of water infiltration and absorption.

5.2.6 Final Model Considerations

As we mentioned at the start of this section, our mechanical model is heavily based on the *Discrete Element Method*. Now that we have explained our method we can explain the differences that our model has with the standard DEM.

The standard DEM models dynamic systems solving differential equations using Newton's second law of motion. In essence, this means that in order to compute the force F_i and moment M_i of each cell (or element using more general terms), the following laws are used:

$$m_i \frac{d^2 s_i}{dt^2} = F_i \tag{5.37}$$

$$I_i \frac{d^2 \theta_i}{dt^2} = M_i \tag{5.38}$$

Where I_i is the second-order moment of inertia tensor. In order to compute these equations, some integrator scheme is used, which implies dynamically updating all forces, moments, positions, and orientations for tiny timesteps Δt. In our case, we discarded the dynamic simulation, and we considered the terrain as static, focusing just on the state of the system at rest. However, we are also interested in the changes in the terrain, so our model will be *Quasi-static* instead of fully static, as we will be jumping from stable states into stable states.

Besides this change from dynamic to quasi-static, the rules that describe the forces and stresses being applied to each link remain the same (or mostly the same) as other DEM models, specially bonded DEMs, as they model materials using a set of grains connected by virtual beams and springs.

5.3 Water Infiltration Process

After computing the decomposition, running the mechanical model, and obtaining the initial state of the system, we are ready to manipulate the terrain using the erosion process. The water infiltration algorithm will be very similar to the original one proposed by [1], with a few necessary changes to adapt it to our mechanical model.

As mentioned before, the idea is that an initial amount of water starts from an exterior link, and then it jumps from link to link following a *stochastic path*. For every link it goes through, some water is absorbed, and the *link is damaged*. When enough links are broken and cells are divided into disconnected components, then *cell detachment* is produced. We are going to explain this process part by part.

5.3.1 Water Paths

First, we start a water path by choosing an external link, which will initially be links that connect solid cells to air cells (it can change with updates on the simulation), using a random distribution. For a given external link e_i, the probability $p_{start}(e_i)$ of being chosen will be set by:

$$p_{start}(e_i) \propto A_i max(n_i \cdot \omega, 0) \tag{5.39}$$

Where A_i is the area of the face connected by e_i, n_i is its normal and ω is a direction chosen by the user. Parameter ω basically represents the direction where water is infiltarting the terrain; it can represent the direction of rainfall, the presence of the sea or lakes at some point of the terrain borders, etc. With the initial link chosen, the water path then selects a new candidate to jump to, following another random distribution. Given an internal link l_i, the probability of jumping to another link $l_j \in \mathcal{N}(l_i)$ will be defined by:

$$p_{flow}(l_i, l_j) \propto max(\vec{g} \cdot \frac{c_j - c_i}{||c_j - c_i||}, 0)(1 - \lambda_j \rho(c_j)) \tag{5.40}$$

Where c_i is the centroid of face l_i and c_j the centroid of l_j, $\lambda_j = 1 - D_j$ is the life of link l_j and, finally, $\rho : \mathbb{R}^3 \to \mathbb{R}$ is a resistance field set by the user. The dot product of gravity and the direction defined by $(c_j - c_i)$ assigns higher probabilities to jumping directions aligned with gravity, and makes it impossible for the water to go "up", or against gravity. The other term assigns higher probability to jump to eroded links, this is because when water infiltrates through the terrain, it makes further infiltration easier. In some situations, we can find that there is no neighbor l_j with $p_{flow}(l_i, l_j) > 0$ (for example when we reach the bottom of the terrain and every jump would imply going against gravity), in that case we stop the path. The other situation were we can stop the water path is if all water has been absorbed by the links.

We start every path with w_{in} water, which will be a fixed quantity, then every link will absorb a quantity of water that will depend on its state. More specifically, the water absorbed by link l_i, $w_{abs}(l_i)$ will be defined as:

$$w_{abs}(l_i) = \begin{cases} A_i k_{solid} + (1 - \lambda_i \rho(c_i)) k_{res} & if \;\; \lambda_i > 0 \\ A_i k_{air} & Otherwise \end{cases} \tag{5.41}$$

Where k_{solid}, k_{res} and k_{air} are constants that control the behavior of water absorption for solid links, the increase on absorption due to erosion and the amount of water that is lost going through broken links. We need to scale absorption by the area A_i, as the bigger the face, the more opportunity there is

for water to be trapped there. Of course, once water is absorbed, we modify the remaining amount of water before jumping to the next link:

$$w'_{act} = w_{act} - w_{abs}(l_i) \tag{5.42}$$

Where w_{act} is the amount of water before the absorption and w'_{act} is the new amount of water after the absorption. Finally, we also include the possibility of a *full absorption* event, where w_{abs} becomes w_{act} and all water is absorbed by the link. The probability of this event happening is proportional to the remaining fraction of water $1 - \frac{w_{act}}{w_{in}}$.

5.3.2 Updating Links

After a water path crosses a link, the link absorbs part of the water, but it has to be damaged accordingly. The criterion used to modify link's life λ after water absoprtion in [1] was the followiong:

$$\lambda'_i = \lambda_i - k_{dmg}\frac{w_{abs}(l_i)}{A_i} \tag{5.43}$$

Where k_{dmg} is a parameter controling how much damage is produced after absorbing a certain quantity per unit area. The idea of this equation works, but we can add the link stresses to extend it. The main idea is that a link under tension lets water pass through more easily, and it is more vulnerable to damage produced by water infiltration, while a link under compression initially produces the opposite effect, not letting water pass through easily and protecting against erosion; however, if the compression stress is too high, it can produce microfractures on the rock that weaken the link. Regarding shear stress, the effect is similar to tension; the closer the stress is to the limit, the weaker the link will be.

Putting it all toghether, the damage applied on link l_i will be affected by modifiers due to the normal and shear stresses:

$$\lambda'_i = \lambda'_i - k_{dmg}\frac{w_{abs}(l_i)}{A_i}(M_n + M_s) \tag{5.44}$$

Where M_n and M_s are the modifiers produced by normal and shear stresses respectively. The normal stress modifier will depend on wether the link is under tension or compression, so we will have 2 equations for M_n that should have the same value when $\sigma_n = 0$ (to preserve continuity).

For the tensile part of M_n, when $\sigma_n > 0$, we propose an equation based in Griffith's brittle fracture theory [37]. The link weakens exponentially under tension, so we define M_n as an exponential:

$$M_n = e^{\alpha_t(\frac{\sigma_n}{T_{max}(D)})^p} \tag{5.45}$$

Where α_t controls how sensitive the rock is to tension (how much tension facilitates damage) and p controls the *brittleness*, higher values of p makes the modifier smaller but with sudden growth when approacing the tension limit. Notice how, regardless of the chosen parameters, $M_n = 1$ when $\sigma_n = 0$, as $e^0 = 1$, so the compression criterion that we use also needs to evaluate to 1 at $\sigma_n = 0$. In addition to evaluating to 1 under no stress, the modifier should also get smaller when reducing σ_n, as compression provides some protection against damage. However, after σ_n surpasses a (negative) threshold, microfractures due to compression are produced and the modifier should increase again. In order to represent this phenomenon we can use the following equation using $x = \frac{|\sigma_n|}{UCS(D)}$:

$$M_n = \frac{1}{1 + sx - \alpha_c x^2} \tag{5.46}$$

Notice how the denominator is 1 when $x = 0$ (so $\sigma_n = 0$) and then as we increase compression, the denominator starts to grow controlled by parameter s (as the squared term grows slower at first). Then, after $sx - \alpha_c x^2$ reaches its maximum, the denominator starts to decrease in value, making M_n grow bigger. We will have to tune s and α_c to properly control the amount of protection given by compression and the threshold where microfractures appear, making the modifier start growing hyperbolically. Putting our two equations toghether, we can define M_n as:

$$M_n = \begin{cases} exp(\alpha_t(\frac{\sigma_n}{T_{max}(D)})^p) & if \;\; \sigma_n > 0 \\ \frac{1}{1+sx-\alpha_c x^2} & Otherwise \end{cases} \tag{5.47}$$

Finally, we will use the ratio between the current shear stress τ and the maximum allowed shear stress $\tau_{limit} = c(D) - \sigma_n tan(\phi)$ provided by the Mohr-Coulomb criterion to define our shear modifier M_s:

$$M_s = \alpha_s(\frac{\tau}{\tau_{limit}})^q \tag{5.48}$$

Similarly to the tensile modifier, α_s will control how much damage is produced on a link under shear stress, and q will control how suddenly or progressively the modifier will grow. Higher values of p will start with a lower value but they will produce a sudden growth when the ratio approaches 1.

As a final note, notice how $0 \leq M_s \leq \alpha_s$ and, under tension, $1 \leq M_n \leq e^{\alpha_t}$ with p and q controling how sudden is the transition from the minimum to the maximum. On the other hand, M_n under compression is harder to bound, and we need to carefuly choose s and α_c to avoid an asimptote when $1+sx-\alpha_c x^2 = 0$. Additionally, all failure limits tend to 0 as D increases, which could produce an asymptote; however, the link will be broken way before the limits reach 0, so we do not have to worry about that.

5.3.3 Cell detachment

Now, the only step left to implement is the cell detachment protocol. After a link has been broken, we do not remove it from the link graph, as water can still go through the broken link, but we do remove the link from the cell graph. After link l_k connecting cells C_i and C_j has been broken, we update the cell neighboring information and compute $\mathcal{N}(C_i)'$ and $\mathcal{N}(C_j)'$ as [3]:

$$\mathcal{N}(C_i)' = \mathcal{N}(C_i) \setminus \{C_j\} \tag{5.49}$$

$$\mathcal{N}(C_j)' = \mathcal{N}(C_j) \setminus \{C_i\} \tag{5.50}$$

Notice how this removal might make cells C_i and C_j completly disconnected, making it impossible to reach C_j from C_i by any path and vice versa. If this happens, it means that the rupture of the link has divided the terrain into two separated components. Conceptually this separation implies that a group of rocks has been disconnected from the terrain, and we will have to erase said group of rocks.

In order to distinguish the *terrain* from the rock that has been separated, we will first look at the core cells. A component that contains a core cell cannot be erased, so if just one of the components contains

[3] Technically, in the implementation we are not going to fully remove the adjacency, just mark it as broken. That is because we are interested on keeping broken links for some purposes

core cells, then the other component will be erased, but if both components contain core cells, then we leave both groups there. Then, the idea proposed by [1] was that if none of the groups had core cells, then we should remove the component with less cell count. We technically keep the same idea, but core cells are mandatory for our mechanical model, so a situation where none of the two components have a core cell can never happen.

Once a component has been separeted, we will need to perform some tasks to keep everything consistent. First, we will mark the removed cells as air, and we will remove them from the mechanical system, recomputing all positions and forces of the new structure without the removed chunk. Then, we will also delete all links that go from an air cell to an air cell, as they should not be accessible anymore. Moreover, if an external portion of the terrain is removed, then some interior cells will now be exposed to the exterior and they will become external, making the links connecting them with the removed part exterior links. Recomputing the exterior links and cells is a bit tricky, as simply marking all cells adjacent to air as exterior does not work (we could have air pockets inside the terrain!), but we can do it the following way: We pick a random exterior link [4] and perform a graph traversal algorithm (like BFS or DFS) jumping to only the neighbors that are adjacent to an air cell; then, we mark all cells travered during the algorithm as exterior (and the links connecting them to an air cell as exterior links). Finally, if our exterior links have changed, we will have to update the probabilities described in equation 5.39.

5.4 Final Algorithm Summary

To end this chapter, after describing every step of the algorithm in detail, we will give step-by-step summary of the method to put it all toghether.

First we start with a model decomposed into voronoi cells, obtained using the method described in chapter 4, and using this decomposition we compute the system of equations described in 5.21 to obtain the displacements of all solid cells. Using said displacements, we obtain the force being applied on each link F_{ij} using 5.10 and then the corresponding stresses with 5.29 and 5.30. We break any stress that passes the failure conditions and, when a link breaks, recompute everything until the system is stable.

Once we have a stable system, we can proceed to the water infiltration protocol. We select an exterior link using 5.39, and from there we start a water path with an initial amount of water w_{ini}. The water path will jump from link to link using 5.40, reducing the amount of water by 5.41 and damaging the link by **??**. If the damage applied to the link makes it break, we update the cell adjacency information with 5.49, and if the graph becomes disconnected, then we update the terrain using the method described in section 5.3.3. Additionally, we will have to repeat the process of computing the mechanical model (described in the previous paragraph) every time a link breaks, even if it does not cause cell detachment.

Finally, a water path will stop once there is no amount of water left ($w'_{act} = 0$ after updating the amount of water) or when there are no more links to jump to ($p_{flow}(l_i, l_j) = 0$ for every $l_j \in \mathcal{N}(l_i)$ or $\mathcal{N}(l_i) = \emptyset$).

[4] Scenarios where removing a chunk of rock implies removing all exterior cells are incredibly rare and we do not contemplate that case.

Chapter 6

Implementation and Results

All methods developed in this project have been implemented in a QT application that allows for testing and debuging the process. In this section we are going to explain the structure of the application, describing which features are implemented and how can the user control different parameters of the method. additionally, we are going to show different results of our techniques, comparing results with different parameters and providing some benchmarks.

All code used for this project can be found on the Github page of the project, for everyone to check and test. It is important to state that we reused the code from a previous project [2] (made by the tutor of this thesis). This previous project provides tools for terrain analysis and synthesis, so it has all tool that we need to load and work with terrains represented as DEMs. Many of the tool provided by the original code are irrelevant to our purposes, so we removed many functionalities; however, we left some tools for terrain analysis as they still provide some utility for debugging.

Finally, before starting this section, all tests have been performed INCLUDE SYSTEM SPECIFICATIONS!!!

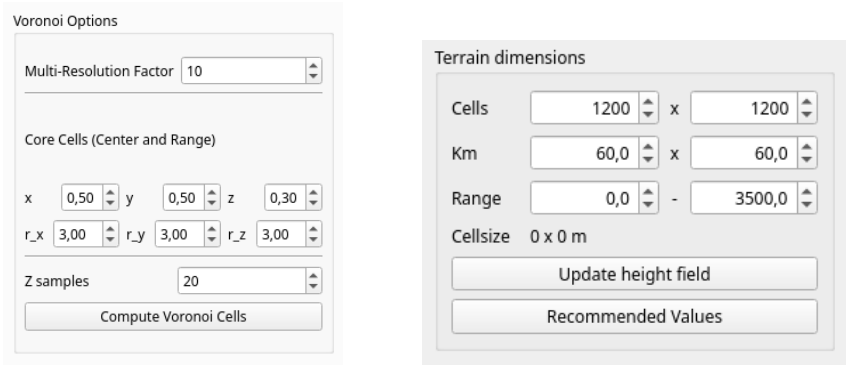
6.1 Voronoi Decomposition

Taking a terrain given by a DEM and converting it to a collection of voronoi cells representing the same (or roughly the same) surface is one of the core aspects of this thesis. We have tested multiple methods and tried multiple parameters to get the most quality posible with resonable efficiency. In this section we are going to describe the tools we give to the user to experiment with the decomposition as well as an analysis and comparison of different methods and parameters tested.

6.1.1 User Interaction

Voronoi Generation Options

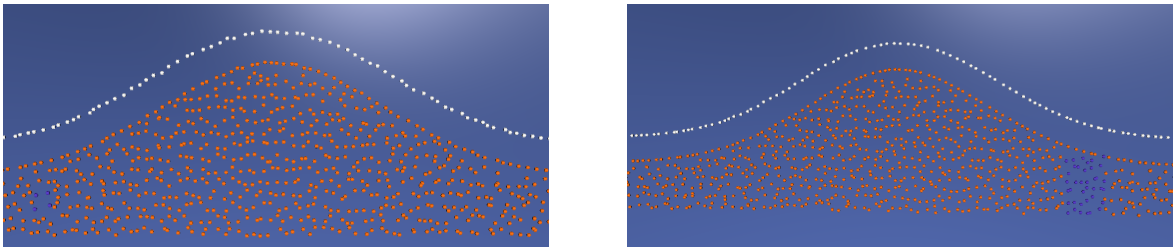
First of all, the user has the option to load heightfields into the application using either ASCII files or pictures in png format.Additionally, we have also provided some *toy examples* that the user can load in order to test all algorithms and see extreme examples. The toy heightfields are designed so it is easy to see the process of both the decomposition and the erosion simulation and therefore they are also small. Most tests have been performed on these toy examples and we left the real terrains for some final simulations, as the final erosion algorithm can be quite heavy on bigger and more detailed terrains.



(a) Voronoi options section

(b) Terrain dimensions section

Figure 6.1: Voronoi options and Terrain dimensions windows of our applicaiton. Both windows can control the results of the decomposition



(a) Core cells when using center $(0.2, 0.2, 0.1)$ and range $(1, 1, 1)$

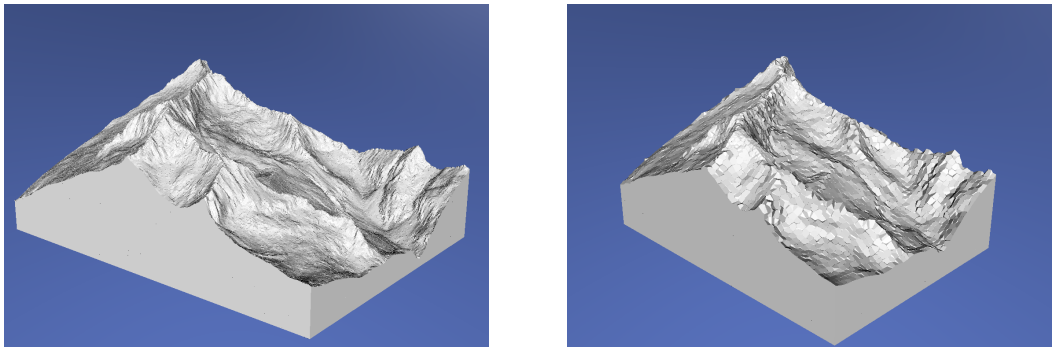
(b) Core cells when using center $(0.8, 0.8, 0.1)$ and range $(3, 3, 3)$

Figure 6.2: Results of modifying the parameters controlling the core cell generation. Particles in purple represent core cells. Figure taken using the *Gaussian Bump* toy heightfield

Once a heightfield is loaded into the application, the user has multiple options to modfiy the resulting voronoi decomposition. First, the most direct way to control the results of the decomposition is the *Voronoi Options* section (found on the voronoi window on the part of the UI at the right side of the application), as seen in figure 6.1(a). This section has a button to compute the decomposition and also has the option to modify multiple parameters of the process. First, the lower resolution level is always automatically computed using the *furthest points sampling* described in 4.4 and the number of points in the lower resolution layer is the total number of points divided by the *Multi-Resolution factor* in the voronoi options.

Second, the user can also determine the generation of core cells (that will not be eroded during the erosion algorithm). A rectangular block of core cells is always generated at a position specified by the user with a size specified by the user. The three parameters (x, y, z) are numbers in the range $[0, 1]$ and define the *center* of the core cell block, the parameters are normalized using the bounding box of the terrain so $(0.5, 0.5, 0.5)$ represent the center of said bounding box. Next, (r_x, r_y, r_z) define the *range* of the core cell block. In order to normalize those values, they measure the range in *cell sizes*, that is, $r_x = 2$ represents a value of 2 times the size of a grid cell in the x dimension. When performing operations with the bounding box of the terrain, keep in mind that the bounding box of the decomposition extends a little bit beyond the highest peak in the z direction, so that we can take samples outside of the terrain.

Finally, the user can also modify the amount of samples used for the decomposition using two options.



(a) Salenques mountain sampled using a 256x184 grid (with 20 vertical samples)

(b) Salenques mountain sampled using a 64x46 grid (with 20 vertical samples)

Figure 6.3: Results of modifying the heightfield dimensions on the terrain *Salenques*

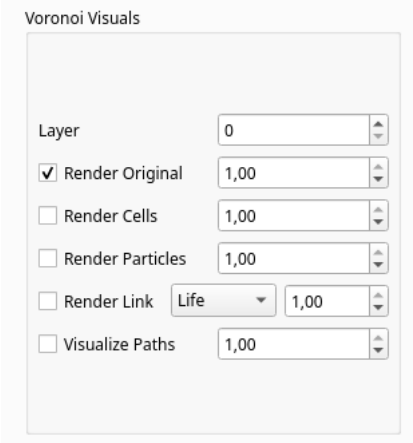
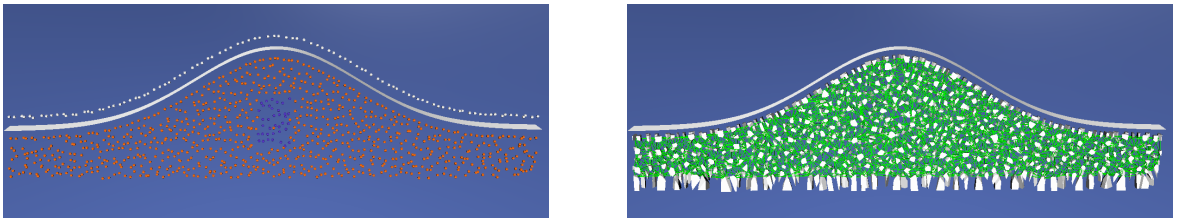


Figure 6.4: Voronoi Visuals section of the UI options

The *Terrain Dimensions* section on the left side of the UI (figure 6.1(b)) allows the user to modify the size of the heightfield. Because we are taking multiple samples for each cell of the heightfield, modifying the terrain dimensions will also modify the number of samples taken. Additionally, in the *Voronoi Options* section, we also can modify the *zSamples* value, which will modify the number of vertical blocks of the 3D grid we construct to sample points. As a final note, there is a button in the *Terrain Dimensions* section called *Recommended Values* that, for bigger terrains, will cap the dimensions (maintaining aspect ration) so they are feasible to compute.

Visualization Options

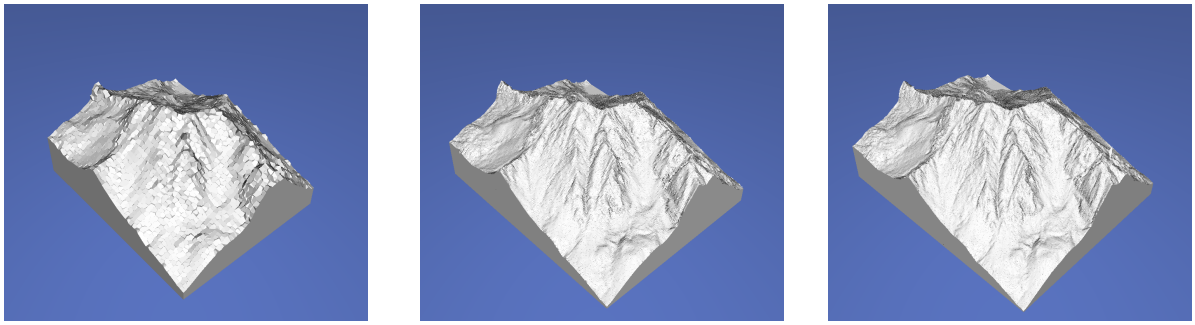
In addition to the decomposition options, we also provided some options to visualize the results of said decomposition. Just above the *Voronoi Options* section, there is a *Voronoi Visuals* section (figure 6.4). Using these options we can visualize the original surface of the terrain, the decomposed terrain made of cells, a visualization of the particles used to make the voronoi cells, a visualization of the connections between cells (and their state, which will be useful when debugging the erosion simulation part), and a visualization of most recently generated water paths (also useful for the erosion part). The user can select multiple options to view multiple representations simultaniously (like visualizing the original surface along other things, as seen in figure 6.5), and there is a variable next to each render option to modfiy the



(a) Example of the particle rendering mode

(b) Example of link rendering

Figure 6.5: Examples of diverse rendering options of our application. REMAKE THIS FIGURE



(a) Punta alta using a 50x50 grid

(b) Punta alta using a 200x200 grid

(c) Punta alta using a 350x350 grid

Figure 6.6: Results of modifying the heightfield dimensions on the terrain *Punta Alta*

opacity of each of them.

6.1.2 Decomposition and Sampling Size

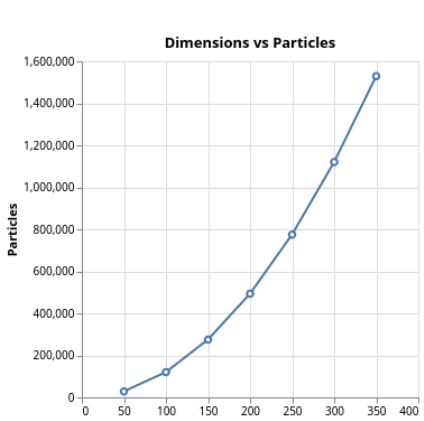
In order to analyze our decomposition method, we performed multiple experiments varying the sampling size and the sampling method used to create the decomposition. By doing these tests, our intention was to better understand the relationship between the computational cost and the quality of the results, as well as obtaining a set of parameters that provide a good balance between cost and quality.

Heightfield Dimensions Experiment

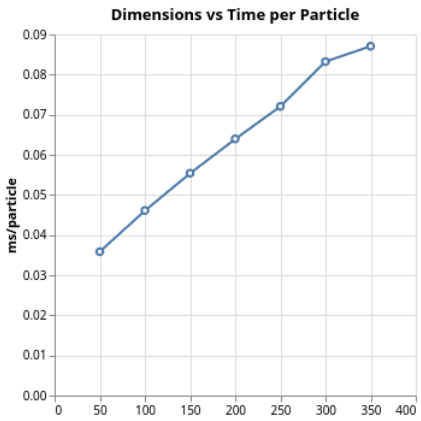
Our first experiment tested the relationship between the heightfield dimensions (the base 2D grid) and the final results, both in terms of computation time and visual appearance. For this experiment we disabled cell subdivision that is produced on high rugosity cells, as changing the terrain dimensions might also have an effect on the rugosity and we wanted to guarantee that the only change on the model are the heightfield dimensions. We also only used the terrain *Punta Alta*, to avoid encountering differences between terrains.

We tried multiple sizes, but the results of the minmum, maximum and median tested dimensions can be appreciated in figure 6.6. The terrain sampled using a 50x50 is really different from the other two, but the 200x200 does not have that much difference with the highest resolution tested, so it provides a good trade-off between efficiency and quality.

Regarding the computational cost, if we consider a square heightfield NxN, the number of particles generated is, of course, $O(N^2)$; however, the time required to compute is not as easy to obtain. In

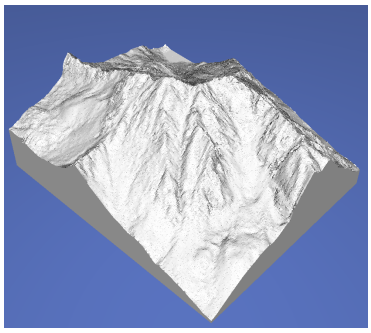


(a) Number of Dimensions vs Number of Particles

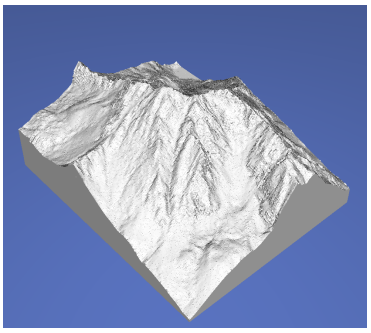


(b) Number of Dimensions vs Time spent per Particle

Figure 6.7: Results of modifying the heightfield dimensions on the terrain *Punta Alta*



(a) Results using $zSamples = 10$



(b) Results using $zSamples = 99$

Figure 6.8: Results of modifying the variable *zSamples* on the terrain *Punta Alta*

the worst case scenario, comptuing a voronoi cell requires intersecting one plan for every other particle, so, the computational cost would be $O(N^3)$. In practice, the library voro++ [3] implements this very efficiently, so this high theoretical cost might not appear in practice. In our empirical test, we found that the number of particles increased quadratically with the dimension N, but we also found that the time spent per particle also increased when increasing dimensions, so we can conclude that the relationship between number of particles and time is not fully linear. This results can be seen in figure 6.7. As a final note, keep in mind that some aspects of the sampling depend on the terrain, as we avoid sampling outside the terrain (so the number of samples in the z direction change depending on the coordinates (x, y)), so the number of particles is not just kN^2 for some constant k.

Changing the Samples in the z-direction

So far we only changed the dimensions of the heightfield, which is a 2D-grid. However, we sample points on a 3D grid, defined by the heightfield dimensions and the variable *zSamples*. We also experimented with this variable and the results have been a little bit surprising. First, the main factor for the visual quality of the decomposition is the accuracy at the surface, as anything below that can not be seen until we perform erosion, which implies that increasing the amount of samples in the z direction would be innitially unappreciable (as seen in figure 6.8).

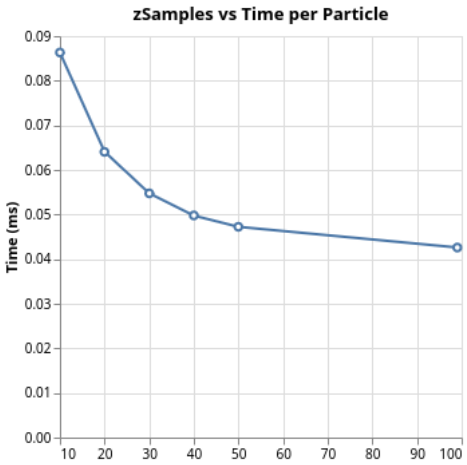
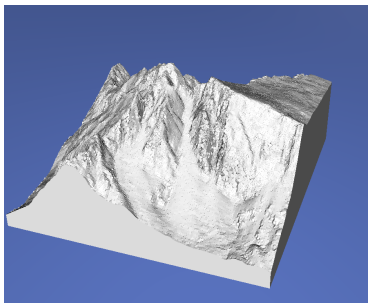
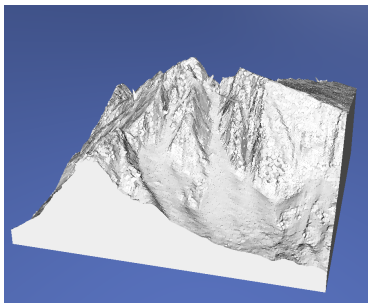


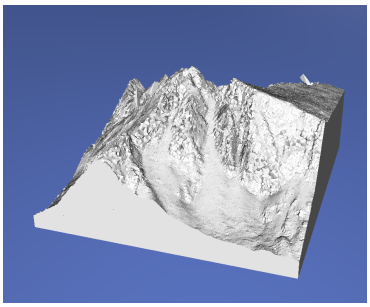
Figure 6.9: Number of samples in the z direction vs computation time spent per particle



(a) bassiero terrain using a 151x138 grid. Subdivision performed for cells with rugosity > 1.5



(b) bassiero terrain using a 151x138 grid. Subdivision performed for cells with rugosity > 2.0



(c) bassiero terrain using a 151x138 grid.Subdivision performed for cells with rugosity > 3.0

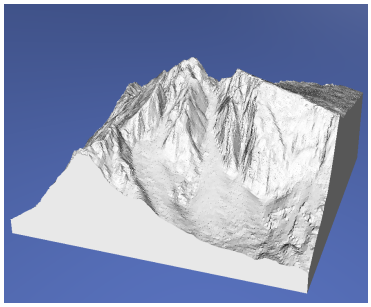
Figure 6.10: Results of changing the rugosity threshold on the *bassiero* terrain

The neglegible effect on visual results was expected, but the unexpected part comes from the time spent to perform the computation. From the results that we got trying different values of *zSamples*, we extracted the time spent per particle, and we observed that this value decreases as we increase the number of samples in the z direction (and thus the number of particles), which is exactly the opposite of what happened when increasing the dimensions of the heightfield. We hypothesise that the particles on the surface are the ones with more computational cost, so increasing the number of samples in the z direction reduces the proportion of particles with high complexity.

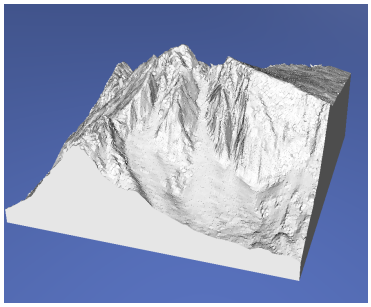
Experimenting with Rugosity Thresholds

For our final test of the terrain decomposition, we reactivated the cell subdivision and we experimented with different rugosity thresholds for subdivisions. First, we tested only perorming one subdivision changing the rugosity threshold (all rugosities are mesured using a window size of 8). The results of this experiment can be seen in figure 6.10.

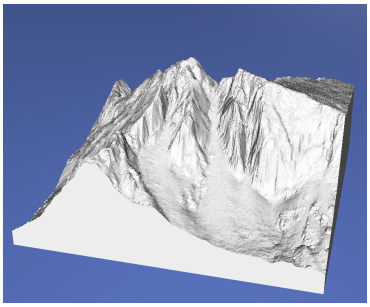
Then, because we are not limited to subdividing a cell only once, we experimented setting two or even three subdivision thresholds. That implies that we will collect $2 \cdot 4^k$ per cell at the surface, where k is the number of thresholds surpased on that specific cell. In this test we tried setting the threshold combinations $\{1.5, 2.0\}$ (the one definitively implemented in the application), $\{2.0, 2.5\}$ and $\{1.5, 2.0, 2.5\}$.



(a) Bassiero terrain using a 151x138 grid. Subdivision performed for cells with rugosity > 1.5 and further subdivision for cells above 2.0



(b) Bassiero terrain using a 151x138 grid. Subdivision performed for cells with rugosity > 2.0 and further subdivision for cells above 2.5



(c) Bassiero terrain using a 151x138 grid.Subdivision performed at three levels with thresholds 1.5, 2.0 and 2.5

Figure 6.11: Results of changing the rugosity threshold on the *Bassiero* terrain. In this case, we perform further subdivisions when the rugosity is too high.

Thresholds	Particles Generated	Time (s)
Base (No subdivisions)	231713	12.639
1.5	293204	16.719
2.0	255445	14.180
3.0	233918	12.844
1.5 & 2.0	388023	22.715
2.0 & 2.5	285736	16.147
1.5 & 2.0 & 2.5	509009	32.446

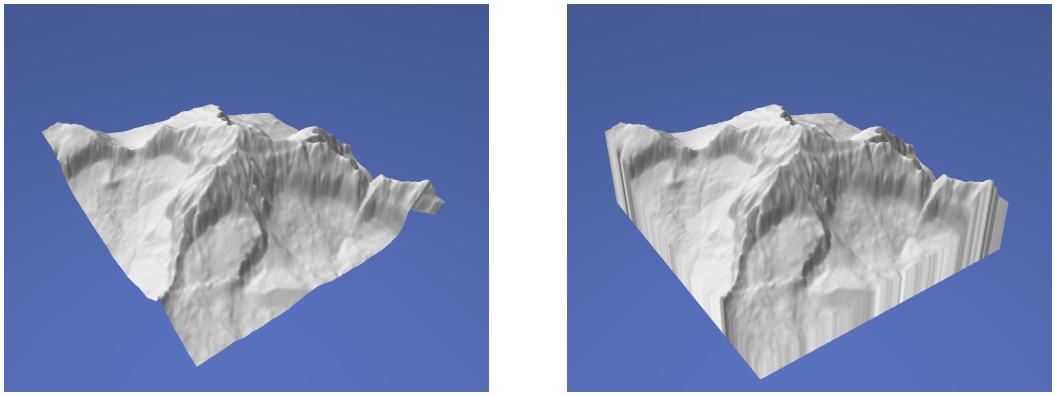
Table 6.1: Measures of our tests with different rugosity thresholds

There is a marked difference between the first and the second test, while the difference between the first and the last (which is the one with highest quality) is more subtle, and only appreciated in the sections with highest rugosity.

Finally, we also measured the number of samples generated for every test, as well as its computation time. Table 6.1 shows all measures we have taken for this batch of experiments. Remember that the computation time is not linear with respect to the number of particles, so it is important to choose a good balance between time spent and visual quality. In our case, the final application that the user can test uses the threshold combination $\{1.5, 2.0\}$, as we found that it provides a good balance.

Different Lighting Test

The lighting method the we use highlights the borders between cells, as each face has a different normal. The lighting method chosen is useful to debug, but it can reduce the visual quality in some situations. As an experiment, we decided to reuse the same lighting method used to render the base heightfield (the one that the base application used) to properly compare the base model with its decomposition. Figure 6.12 show the results of this experiment: the terrain looks basically the same except for the walls (generated during th decomposition). It is important to note that the lighting method used assigns a color based on the position of a point in the 2D grid of the DEM, so all points with the same (x, y) coordinates will have the same color.



(a) Base Aneto terrain

(b) Aneto terrain after the decomposition

Figure 6.12: Comparison of the base terrain and the decomposition result

Multi-Resolution factor	Particles Generated	Time (s)
Base (single resolution)	411635	22.813
Furthest Point: 3	137211	283.913
Furthest Point: 5	82327	183.684
Furthest Point: 10	41163	103.141
Furthest Point: 20	20581	64.237
Grid: 2x2x2	26208	25.673
Grid: 3x3x3	10294	24.210
Grid: 4x4x4	5398	23.775

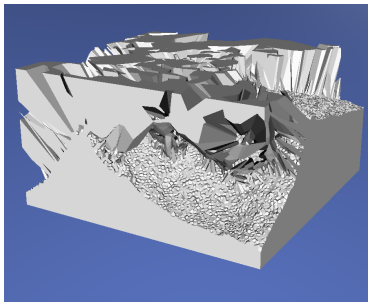
Table 6.2: Comparison between the two subsampling methods that we implemented

6.1.3 Multi-Resolution Comparison

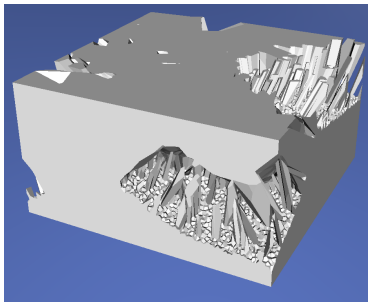
In addition to all tests regarding the initial decomposition, we also compared our two proposed methods for subsampling. The two methods are hard to compare, as the method of subsampling in a grid does not allow for an easy manipulation on the total size of the subsampling (The grid that we use to sample and subsample is terrain dependent). Despite this limitations, we decided to take some measures for each method in order to perform some comparisons.

First, tested both the time to compute and the final result of the furthest point sampling using N/s subsamples where $s \in \{3, 5, 10, 20\}$, and then, we tested the subsampling method based on a grid using 2x2x2, 3x3x3 and 4x4x4 blocks of grid cells. The visual results can be appreciated in figures 6.13 and 6.14 and the computation time of each situation is written in table 6.2. All times are computed from the start of the decomposition to the completion of the subsample, so they all include the base time to produce the initial sampling.

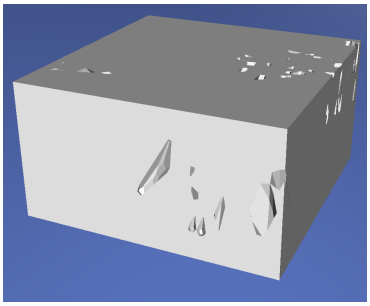
Even if these two methods are hard to compare, we can see that, with a similar amount of particles (using a downsampling factor of 20 vs a 2x2x2 grid), the grid method is much more efficient in producing the computation, and also produces shapes that more closely resemble the base terrain. Additionally, the examples that use the grid method add very little overhead over the initial sampling of the decomposition, if we substracted the time of the initial computation to all total computation times, we will see a sharper difference between the furthest point and the grid methods.



(a) Bassiero terrain subsampled using furthest point with a downscale factor of 3

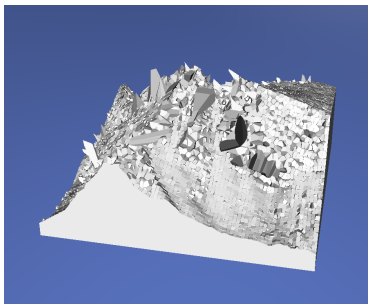


(b) Bassiero terrain subsampled using furthest point with a downscale factor of 5

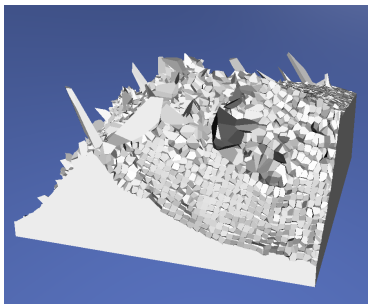


(c) Bassiero terrain subsampled using furthest point with a downscale factor of 10

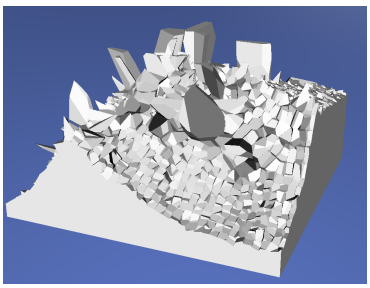
Figure 6.13: Different downscaling factors for the furthest point sampling method applied to the *Bassiero* terrain.



(a) Bassiero terrain subsampled using a subgrid made of 2x2x2 blocks



(b) Bassiero terrain subsampled using a subgrid made of 3x3x3 blocks



(c) Bassiero terrain subsampled using a subgrid made of 4x4x4 blocks

Figure 6.14: Comparison of different block sizes used to make a subsampled gird. Applied to the *Bassiero* Terrain.

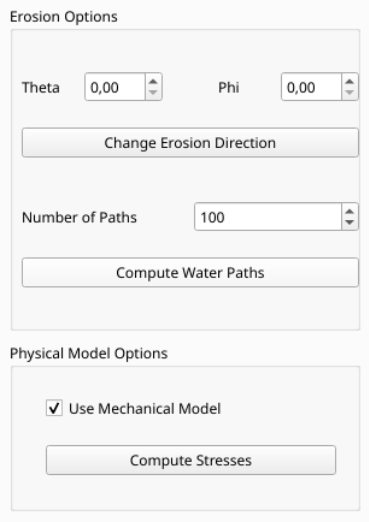


Figure 6.15: Erosion menu of the UI

6.2 Erosion Simulation

In this project we deeply tested the erosion simulation algorithm, trying different paramaters, different decompositions and comparing the old algorithm versus the new one. In this section we are going to go over our implementation of the erosion process as well as explaining all our findings over our experiments.

6.2.1 UI For Erosion

Before explining our findings, we are going to show how the user can control the different aspects of the simulation. Most parameters, like the material properties are set in the *mechanicalModel.h* header with no option to control them on the UI, but the user still has some freedom using just the UI.

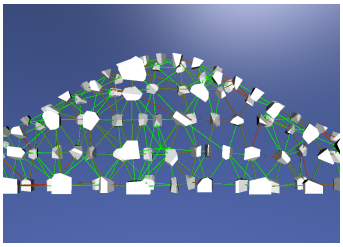
Erosion Menu

From the erosiin menu of the UI (shown in figure 6.15), the user can modify the direction of water infiltration (as explained in section 5.3.1) and wheter the mechanical model is used or not (keep in mind that an initial stress computation will be performed every time you activate the mechanical model option). Even if the user decides to not use the mechanical model, the button *Compute Stresses* computes all stresses of the system and removes the links meeting the failure conditions, this way be can simulate the process without mechanical model up to a point, and then compute the state of the system in that position.

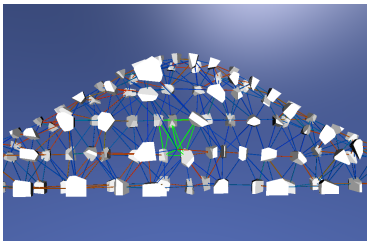
Finally, the user will have to select the number of paths to be generated simultaneously and press the button *Compute Water Paths* to start the simulation.

Visualization Options

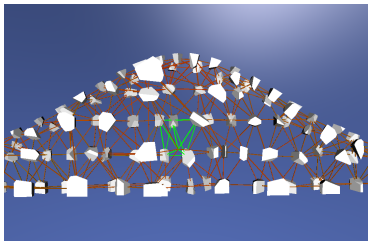
In addition to the options used to control the simulation, the visualization menu 6.4 also contains two options that are useful to debug the erosion process. First, we can visualize the life of the links and their shear and normal stresses using the *Render Links* option (figure 6.16), and second, we can paint blue the cells touched by the last batch of water paths (remember that the size of the batch is controlled using the erosion options), as shown in figure 6.17.



(a) Link life visualization. From Green (full life) to red (fully damaged)



(b) Link normal stress visualization. From blue to green for negative normal stress (compression) and green to red for tension



(c) Link shear stress visualization. From green (0 stress) to red

Figure 6.16: Different link visualization options

Figure 6.17: Simulation of 5 water paths on the *Gausssian Bump* toy example

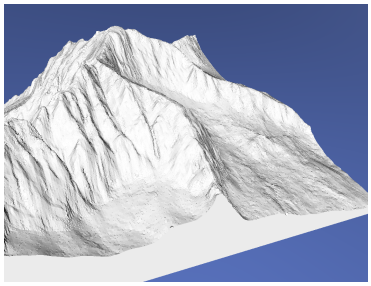
6.2.2 Base Algorithm Experimentation

The first set of experiments that we conducted were all centered on the base algorithm as designed in [1], without the inclusion of the mechanical model. Having a set of experiments on the base method allows us to better compare both techniques and pinpoint the advantadges of adding the mechanical model on top of the base one.

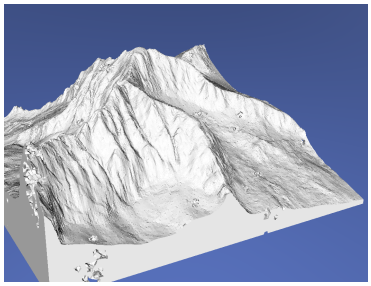
Our first batch of experiments consited in comparing both multi-resolution methods that we implemented to check the difference in results, and additionally take measures of computational time. For these experiments we tested 5 consecutive rounds of 10 thousand paths for each multi-resolution method using the terrain *Aneto*, and then 3 consecutive rounds of 100 thousand paths on a fresh instance of the same terrain. We decided to take multiple rounds of paths instead of just one single round with the combined number of paths to observe the evolution of the terrain step by step and check the influence of the current state of the terrain on the computational cost.

Regarding the final results, figure 6.18 shows the base Aneto terrain compared against the final result of simulating 300 thousand water paths on each multi-resolution method. The furthest point method generated more cells than the grid method (66491 vs 18736), so a slower erosion process was expected. Still, as we mentioned in section 4.4, the furthest point method has to consider some cells outside of the terrain as solid; this means that in order to start appreciating results on the terrain, we will have to wait for the outside cells to be removed, which makes the full simulation slower.

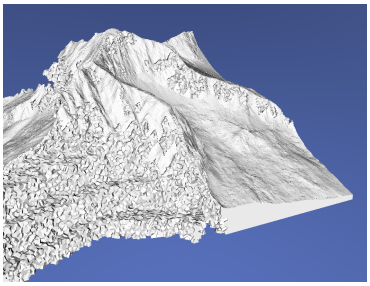
In terms of the computational cost, we have seen that it increased for every consecutive round 10



(a) Aneto terrain before any erosion process

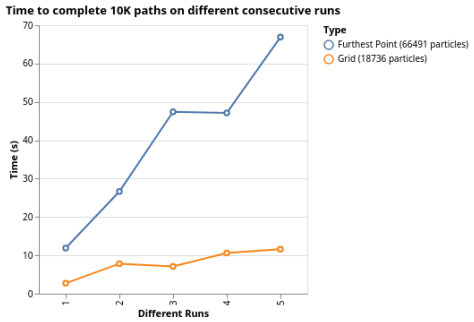


(b) Aneto terrain after 300 thousand paths using the furthest point method to subsample

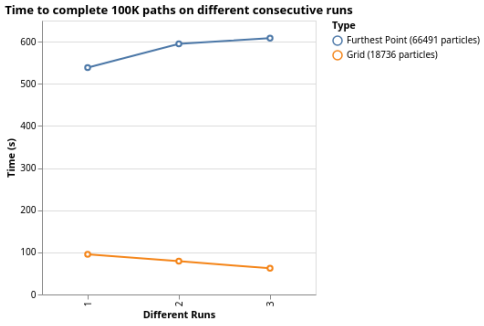


(c) Aneto terrain after 300 thousand paths using the grid method to subsample

Figure 6.18: Result of 300 thousand water paths on the Aneto terrain using different subsampling methods to create the low-resolution decomposition



(a) Time to complete different runs of 10K paths. The runs have been executed one after the other on the same model



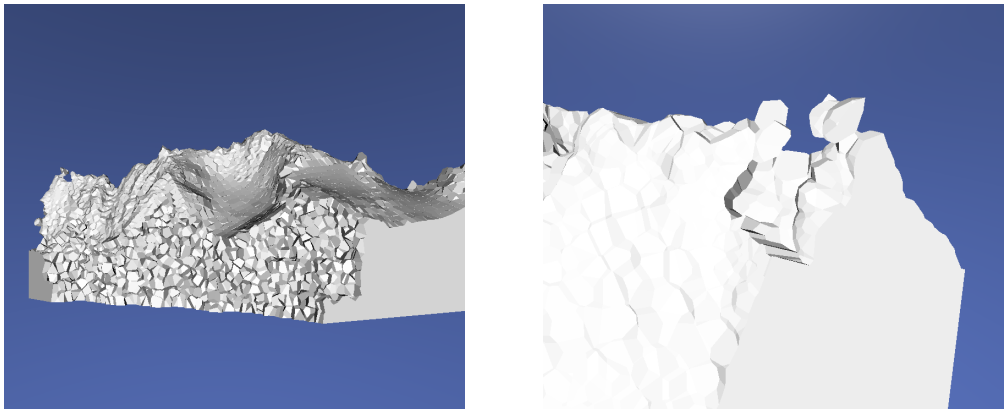
(b) Time to complete different runs of 100K paths. The runs have been executed one after the other on the same model

Figure 6.19: Time to complete of different consecutive runs with 10K and 100K paths for run

thousand paths with both subsampling methods (figure 6.19(a)). We think that the more damaged the links are, the more removal events will be triggered, which are the most coputationally expensive procedures of the algorithm, so the total time to compute will increase. Additionally, the more eroded the terrain is and the less cells remaining, the faster the algorithm should run, as the size of the problem is smaller. Following this logic, there should be a threshold where the cost of each consecutive round starts decreasing instead of increasing, as the size of the terrain starts getting significantlly smaller. Figure 6.19(b), seems to confirm this idea, as the computational cost of the 100 thousand path rounds decreases for each consecutive run in the grid method case. In the case of the furthest point sampling, we have already seen that the erosion process is slower, so we think that 300 thousand paths was not enough to reach the threshold where the time starts decreasing instead of increasing.

6.2.3 Mechanical Model Experimentation

After doing some experimentation with the base model, we are ready to test the algorithm including the mechanical model. The mechanical model is much more computationally expensive than the base algorithm, due to the constant recomputation of the system of equations, so we need a simpler model to test. We used the *Aneto* terrain with heightfield dimensions 50x50 and a grid 3x3x3 to perfeorm all tests on our mechanical model.



(a) Results using the initial cohesion, without modification

(b) Minor damage initial damage on the terrain using the updated cohesion

Figure 6.20: Results of our test runs to tune the material properties

	Granite	Marble	Sandstone
Density	2650 kg/m^3	2700 kg/m^3	2600 kg/m^3
Young's Modulus (E)	70 GPa	54 GPa	38 GPa
Poisson's Ratio (υ)	0.255	0.2	0.25
Tensile Strength (T_{max})	60 MPa	12 MPa	6 MPa
Compressive Strength (UCS)	2.2 GPa	540 MPa	120 MPa
Cohesion (c)	200 MPa	700 MPa	200 MPa
$tan(\varphi)$	0.267949	0.57735026	0.83909963

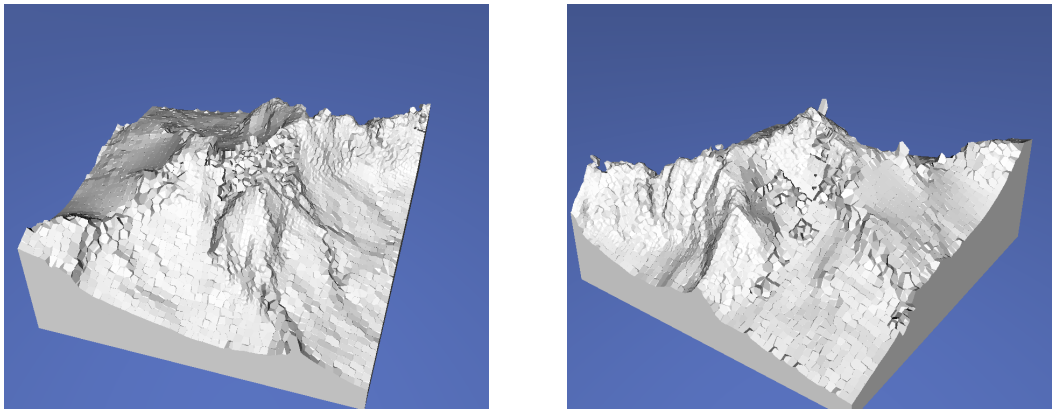
Table 6.3: Parameters of the tested materials. Some properties have been adjusted to our algorithm and might not fully represent the true material properties

Simulating Different Materials

The number of parameters influencing our mechanical model is huge, and independently trying to tune every single parameter would be a challenging task. For that reason we decided to first look in nature and try the mechanical properties of different rocks. We do not really need the parameters to exactly match the real ones, and we can work with an approximation, so we took the data from the website [38], that lists some of the material properties of different materials. However, that website does not list cohesion (c) and the coefficient of internal friction (φ), so we checked for those propoerties in other parts of the literature like [39], [40] and [41].

Using the collected data on material properties, we did a test run for granite, just to check whether the model runs properly or not. In the test run we found that most links were, innitially, over the shear stress limit, producing an initial crack propagation effect that destroyed a huge portion the terrain without starting the first path. For that reason we decided to multiply by 10 the cohesion of all materials, and then we tried again using granite. In the second test run, some links broke initially due to tension and a few cells were separated, but the initial damage was minor. To avoid this initial damage we decided to increase the tensile strength by 50% on all materials. Figure 6.20 shows the results of these two test runs.

After these test runs, we set the material poperties of granite, marble and sandstone to the values shown in table 6.3 and then started some experiments to compare the three. Granite show pretty good



(a) Results of a granite Aneto after 100 water paths

(b) Results of a marble Aneto after the initial stress computation (0 paths)

Figure 6.21: Results of our experiments with different materials

	Reinforced Granite	Reinforced Marble	Reinforced Sandstone
Density	2650 kg/m^3	2700 kg/m^3	2600 kg/m^3
Young's Modulus (E)	70 GPa	54 GPa	38 GPa
Poisson's Ratio (υ)	0.255	0.2	0.25
Tensile Strength (T_{max})	390 MPa	120 MPa	60 MPa
Compressive Strength (UCS)	2.2 GPa	540 MPa	120 MPa
Cohesion (c)	200 MPa	700 MPa	200 MPa
$tan(\varphi)$	0.267949	0.57735026	0.83909963

Table 6.4: Parameters that we used as our *reinforced* materials

results after simulating 100 paths, the simulation took arround 1000 seconds and it produced a big hole in the middle of the mountain (somwhat resembling a volcano). Regarding the other two materials, marble showed some fractures after the initial stress computation (before generating any water path) and then completly collapsed after just 10 paths, while sandstone did not even survive the initial stress computation. We based our parameters on the initial test runs on granite (which is the only one that survived a few paths) but it seems that our takeaways from the test runs were not really applicable to the other two materials.

Reinforced Materials

In response to the poor results of the first experiments with different materials, we decided to reinforce their tensile strenght in order to make them resist at least a few water paths. We decided to adjust just the tensile strenght because most links were breaking due to tension and not due to shear or compression stresses. To differentiate these modified materials with the base materials that we tried previously, we decided to call them *reinforced materials*, with properties shown in table 6.4.

Using these new reinforced materials, our experiments looked much better and we were able to compare the effect of changing the material on the overall results. After 300 water paths on each material, granite produced multiple wide but shallow holes, sandstone produced deep holes with a stair pattern produced by different depths on the hole, and, finally, marble seemed the most resistant, producing thin and shallow holes.

Failure Parameter Tunning

Computational Time Results

6.2.4 Method Comparison

Chapter 7

Conclusions

Summary of the previous results, final conclusions and closing thoughts

7.1 Future Work

7.2 Conclusions

Bibliography

[1] Diego Mateos et al. "Simulation of Mechanical Weathering for Modeling Rocky Terrains ". In: *Spanish Computer Graphics Conference (CEIG)*. Ed. by Julio Marco and Gustavo Patow. The Eurographics Association, 2024. ISBN: 978-3-03868-261-5. DOI: 10.2312/ceig.20241139.

[2] O. Argudo et al. "Terrain descriptors for landscape synthesis, analysis and simulation". In: *Computer Graphics Forum* 44.2 (2025). DOI: https://doi.org/10.1111/cgf.70080.

[3] ChrisH. Rycroft. *VORO++*. Feb. 2009. DOI: 10.11578/dc.20210416.28. URL: https://www.osti.gov/biblio/code-54674.

[4] Eric Galin et al. "A review of digital terrain modeling". In: *Computer Graphics Forum*. Vol. 38. 2. Wiley Online Library. 2019, pp. 553–577.

[5] B. Benes and R. Forsbach. "Layered data representation for visual simulation of terrain erosion". In: *Proceedings Spring Conference on Computer Graphics*. 2001, pp. 80–86. DOI: 10.1109/SCCG.2001.945341.

[6] Guillaume Cordonnier et al. "Authoring landscapes by combining ecosystem and terrain erosion simulation". In: *ACM Trans. Graph.* 36.4 (July 2017). ISSN: 0730-0301. DOI: 10.1145/3072959.3073667. URL: https://doi.org/10.1145/3072959.3073667.

[7] Dennis Wingender and Daniel Balzani. "Simulation of crack propagation through voxel-based, heterogeneous structures based on eigenerosion and finite cells". In: *Computational Mechanics* 70.2 (2022), pp. 385–406.

[8] Giuseppe Turini, Fabio Ganovelli, and Claudio Montani. "Simulating Drilling on Tetrahedral Meshes." In: *Eurographics (Short Presentations)*. 2006, pp. 127–131.

[9] Matias Muller. *100 X simulation speedup with mesh embedding*. 2022. URL: https://matthias-research.github.io/pages/tenMinutePhysics/12-softBodySkinning.pdf.

[10] Mojtaba Mohammadnejad et al. "An overview on advances in computational fracture mechanics of rock". In: *Geosystem Engineering* 24.4 (2021), pp. 206–229.

[11] Ante Munjiza. *The Combined Finite-Discrete Element Method*. Dec. 2004, pp. 1–333. ISBN: 9780470841990. DOI: 10.1002/0470020180.

[12] Vedad Tojaga et al. "Advances in discrete element modeling of rock fracture for next-generation comminution models". In: *Computational Particle Mechanics* 12.6 (2025), pp. 4431–4449.

[13] D.O. Potyondy and P.A. Cundall. "A bonded-particle model for rock". In: *International Journal of Rock Mechanics and Mining Sciences* 41.8 (2004). Rock Mechanics Results from the Underground Research Laboratory, Canada, pp. 1329–1364. ISSN: 1365-1609. DOI: https://doi.org/10.1016/j.ijrmms.2004.09.011. URL: https://www.sciencedirect.com/science/article/pii/S1365160904002874.

[14] Luisa Fernanda Orozco et al. "Discrete-element model for dynamic fracture of a single particle". In: *International Journal of Solids and Structures* 166 (2019), pp. 47–56.

[15] Daniel Cantor et al. "Three-dimensional bonded-cell model for grain fragmentation". In: *Computational Particle Mechanics* 4.4 (2017), pp. 441–450.

[16] Alan Norton et al. "Animation of fracture by physical modeling". In: *Vis. Comput.* 7.4 (July 1991), 210–219. ISSN: 0178-2789. DOI: 10.1007/BF01900837. URL: https://doi.org/10.1007/BF01900837.

[17] Demetri Terzopoulos and Kurt Fleischer. "Modeling inelastic deformation: viscolelasticity, plasticity, fracture". In: *Proceedings of the 15th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH '88. New York, NY, USA: Association for Computing Machinery, 1988, 269–278. ISBN: 0897912756. DOI: 10.1145/54852.378522. URL: https://doi.org/10.1145/54852.378522.

[18] Joshua A. Levine et al. "A Peridynamic Perspective on Spring-Mass Fracture". In: *Eurographics/ACM SIGGRAPH Symposium on Computer Animation*. Ed. by Vladlen Koltun and Eftychios Sifakis. The Eurographics Association, 2014. ISBN: 978-3-905674-61-3. DOI: 10.2312/sca.20141122.

[19] Saty Raghavachary. "Fracture generation on polygonal meshes using Voronoi polygons". In: *ACM SIGGRAPH 2002 Conference Abstracts and Applications*. SIGGRAPH '02. San Antonio, Texas: Association for Computing Machinery, 2002, p. 187. ISBN: 1581135254. DOI: 10.1145/1242073.1242200. URL: https://doi.org/10.1145/1242073.1242200.

[20] Adrien Peytavie et al. "DeadWood: Including Disturbance and Decay in the Depiction of Digital Nature". In: *ACM Trans. Graph.* 43.2 (Feb. 2024). ISSN: 0730-0301. DOI: 10.1145/3641816. URL: https://doi.org/10.1145/3641816.

[21] Bosheng Li et al. "Stressful Tree Modeling: Breaking Branches with Strands". In: *Proceedings of the Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Papers*. SIGGRAPH Conference Papers '25. New York, NY, USA: Association for Computing Machinery, 2025. ISBN: 9798400715402. DOI: 10.1145/3721238.3730745. URL: https://doi.org/10.1145/3721238.3730745.

[22] ZHANYU YANG and NIKOLAS A. SCHWARZ. 2026. URL: https://jango6324.github.io/woodstock/.

[23] T. Ito et al. "Modeling rocky scenery taking into account joints". In: *Proceedings Computer Graphics International 2003*. 2003, pp. 244–247. DOI: 10.1109/CGI.2003.1214475.

[24] Zhanyu Yang et al. "Arenite: A Physics-based Sandstone Simulator". In: *ACM Trans. Graph.* 44.4 (July 2025). ISSN: 0730-0301. DOI: 10.1145/3731201. URL: https://doi.org/10.1145/3731201.

[25] Axel Paris et al. "Terrain Amplification with Implicit 3D Features". In: *ACM Transactions on Graphics* 38.5 (2019). DOI: 10.1145/3342765. URL: https://hal.science/hal-02273097.

[26] Axel Paris et al. "Modeling Rocky Scenery using Implicit Blocks". In: *The Visual Computer* 36.10 (2020), pp. 2251–2261. DOI: 10.1007/s00371-020-01905-6. URL: https://hal.science/hal-02926218.

[27] Axel Paris et al. "Synthesizing Geologically Coherent Cave Networks". In: *Computer Graphics Forum* (2021). DOI: 10.1111/cgf.14420. URL: https://hal.science/hal-03331697.

[28] Guillaume Cordonnier et al. "Sculpting Mountains: Interactive Terrain Modeling Based on Subsurface Geology". In: *IEEE Transactions on Visualization and Computer Graphics* 24.5 (May 2018), pp. 1756–1769. DOI: 10.1109/TVCG.2017.2689022. URL: https://hal.science/hal-01517343.

[29] Éric Guérin et al. "Interactive example-based terrain authoring with conditional generative adversarial networks". In: *ACM Trans. Graph.* 36.6 (Nov. 2017). ISSN: 0730-0301. DOI: 10.1145/3130800.3130804. URL: https://doi.org/10.1145/3130800.3130804.

[30] C. Grenier et al. "Real-time Terrain Enhancement with Controlled Procedural Patterns". In: *Computer Graphics Forum* 43.1 (2024), e14992. DOI: https://doi.org/10.1111/cgf.14992. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.14992. URL: https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.14992.

[31] O. Argudo, A. Chica, and C. Andujar. "Terrain Super-resolution through Aerial Imagery and Fully Convolutional Networks". In: *Computer Graphics Forum* 37.2 (2018), pp. 101–110. DOI: https://doi.org/10.1111/cgf.13345. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.13345. URL: https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.13345.

[32] Diego Mateos Arlanzón. "Simulation of mechanical weathering for modeling rocky terrains". Projecte Final de Màster Oficial. UPC, Facultat d'Informàtica de Barcelona, Departament de Ciències de la Computació, Oct. 2023. URL: https://hdl.handle.net/2117/403778.

[33] Emanuel A. Lazar, Jiayin Lu, and Chris H. Rycroft. "Voronoi cell analysis: The shapes of particle systems". In: *American Journal of Physics* 90.6 (June 2022), pp. 469–480. ISSN: 0002-9505. DOI: 10.1119/5.0087591. eprint: https://pubs.aip.org/aapt/ajp/article-pdf/90/6/469/20100696/469_1_5.0087591.pdf. URL: https://doi.org/10.1119/5.0087591.

[34] Jeff S Jenness. "Calculating landscape surface area from digital elevation models". In: *Wildlife Society Bulletin* 32.3 (2004), pp. 829–839.

[35] Gaël Guennebaud, Benoît Jacob, et al. *Eigen*. https://libeigen.gitlab.io. 2010.

[36] Joseph F Labuz and Arno Zang. "Mohr–Coulomb failure criterion". In: *The ISRM suggested methods for rock characterization, testing and monitoring: 2007-2014*. Springer, 2014, pp. 227–231.

[37] Jian-Ying Wu and Vinh Phu Nguyen. "A length scale insensitive phase-field damage model for brittle fracture". In: *Journal of the Mechanics and Physics of Solids* 119 (2018), pp. 20–42. ISSN: 0022-5096. DOI: https://doi.org/10.1016/j.jmps.2018.06.006. URL: https://www.sciencedirect.com/science/article/pii/S0022509618302643.

[38] URL: https://www.makeitfrom.com/.

[39] Zhixiang Song et al. "Time-dependent behaviors and volumetric recovery phenomenon of sandstone under triaxial loading and unloading". In: *Journal of Central South University* 29 (Jan. 2023), pp. 4002–4020. DOI: 10.1007/s11771-022-5207-2.

[40] Quan Jiang et al. "Coupling Deterioration Model of Mechanical Parameters for the Jinping Marble Under Progressive Damage Conditions". In: *Rock Mechanics and Rock Engineering* 56 (Mar. 2023), pp. 1–26. DOI: 10.1007/s00603-023-03268-5.

[41] Jincai Yu et al. "Strength Parameters and Failure Criterion of Granite After High-Temperature and Water-Cooling Treatment". In: *Applied Sciences* 15.13 (2025). ISSN: 2076-3417. DOI: 10.3390/app15137481. URL: https://www.mdpi.com/2076-3417/15/13/7481.

[42] John P. Wilson. "Digital terrain modeling". In: *Geomorphology* 137.1 (2012). Geospatial Technologies and Geomorphological Mapping Proceedings of the 41st Annual Binghamton Geomorphology Symposium, pp. 107–121. ISSN: 0169-555X. DOI: https://doi.org/10.1016/j.geomorph.2011.03.012. URL: https://www.sciencedirect.com/science/article/pii/S0169555X11001449.

[43] Farès Belhadj. "Terrain modeling: a constrained fractal model". In: *Proceedings of the 5th International Conference on Computer Graphics, Virtual Reality, Visualisation and Interaction in Africa*. AFRIGRAPH '07. Grahamstown, South Africa: Association for Computing Machinery, 2007, 197–204. ISBN: 9781595939067. DOI: 10.1145/1294685.1294717. URL: https://doi.org/10.1145/1294685.1294717.

[44] Mattia Natali et al. "Modeling Terrains and Subsurface Geology." In: *Eurographics (State of the Art Reports)*. 2013, pp. 155–173.

[45] Demetri Terzopoulos and Kurt Fleischer. "Modeling inelastic deformation: viscolelasticity, plasticity, fracture". In: *SIGGRAPH Comput. Graph.* 22.4 (June 1988), 269–278. ISSN: 0097-8930. DOI: 10.1145/378456.378522. URL: https://doi.org/10.1145/378456.378522.

[46] Shiyu Zhao. *Time Derivative of Rotation Matrices: A Tutorial*. 2016. arXiv: 1609.06088 [cs.RO]. URL: https://arxiv.org/abs/1609.06088.

[47] Yogesh Surkutwar, Corina Sandu, and Costin Untaroiu. "Review of modeling methods of compressed snow-tire interaction". In: *Journal of Terramechanics* 105 (2023), pp. 27–40. ISSN: 0022-4898. DOI: https://doi.org/10.1016/j.jterra.2022.10.004. URL: https://www.sciencedirect.com/science/article/pii/S0022489822000726.